

ARCHITECTURE LOGICIELLE: DETAILS

III. Styles d'architecture

III.1 Appels et Retours

III.2 Couches

III.3 Flot de données

III.4 Architecture distribuée

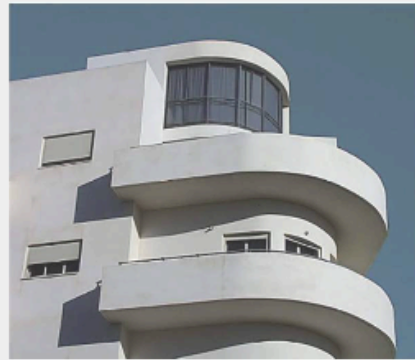
III.5 Architecture orientée évènements

III.6 Architecture centrée sur les données

III.7 Styles hybrides

III. Styles d'architecture

Notion de styles



- A l'instar de l'architecture traditionnelle, l'architecture logicielle peut se caractériser par des **styles**.
- Selon le résultat attendu, un système pourra
 - utiliser *plusieurs styles*
 - ou *imbriquer des styles* les uns dans les autres (*fractilité*)

Découpe générique hors style

Système => Système complet

└─ **Sous-Système** => *service business* (ex: Gestion des comptes)

└─ **Composant** (ex: AccountService.jar)
 => *unité de déploiement*

└─ **Module** => *unité logique* (ex: Account + Transaction)

└─ **Classe | Fonction** (ex: Account, Transaction)

styles principaux

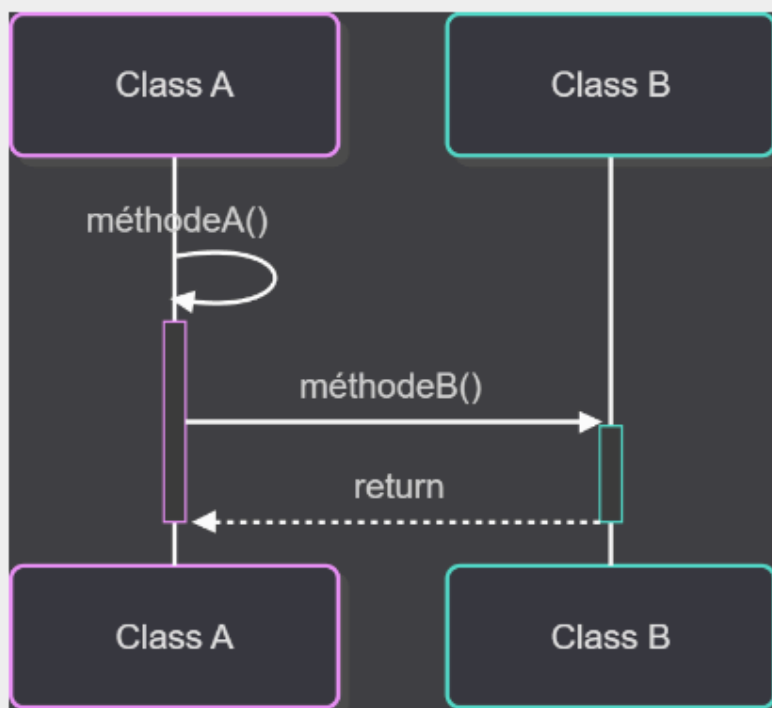
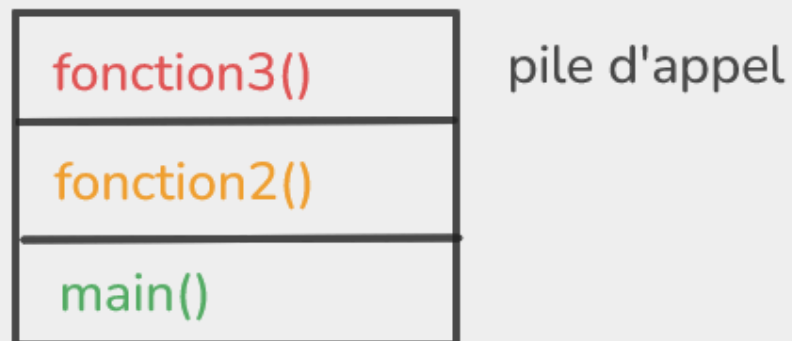
- Architecture en appels et retours
- **Architecture en couches**
- Architecture en flot de données
- **Architecture distribuée**
- Architecture centrée sur les données
- **Architecture orientée objets** (déjà vu)

III.1 Appels et Retours

- Principe de **décomposition fonctionnelle de Niklaus Wirth**
 - décomposer la fonctionnalité initiale en sous fonctionnalités jusqu'à obtenir des problèmes simples
 - principe **K**ee**P** **I**t **S**imple **S**tupid
- Caractéristiques:
 - *synchrone*
 - *hiérarchie d'appels* (pile d'appel LIFO)
- **+++ : diminution de la complexité.**
- **--- : diminution de la performance**

Appels et Retour: exemples

main() # programme principal
 └─ fonction1() # sous-routine
 └─ fonction2()
 └─ fonction3() # imbrication



un service est une unité d'exécution (sous-système)
 exécuté sur un processeur / processus isolé
 et qui communique sur le réseau

un service n'est pas un élément d'architecture

III.2 couches

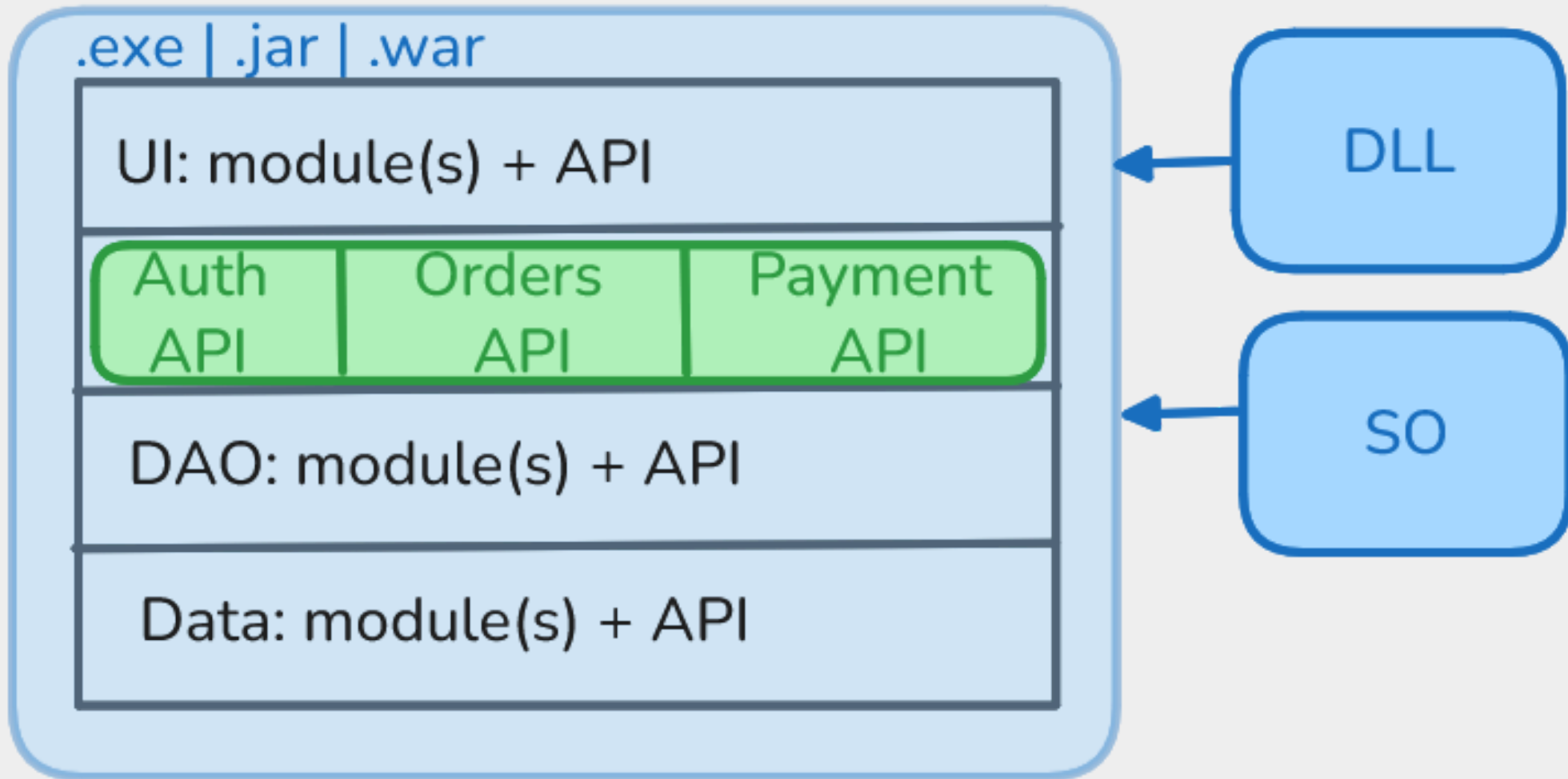
- Le système est décomposé en *couches horizontales*,
 - utilisant le **SRP** pour garantir une **responsabilité spécifique**.
- **Ré-utilisabilité** des couches inférieures par les couches supérieures, par *appels et retours*
 - *jamais dans l'autre sens !*
- **Découpe Technique**: persistance / métier / vue (*3-tiers*) / interlogiciels ... (*n-tiers*)

couches: schéma



disposition des couches: monolithe

- *toutes les couches* sont contenues dans un *exécutable unique*
 - **monolithe**
 - *+++* facile à déployer
 - *---* difficile à faire évoluer
 - **monolithe modulaire**
 - *+++* architecture propre avec périphs en tant que plug-ins*
- “ *ex: application desktop* ”



disposition des couches: client-serveur

- utilisation de services distants

client lourd / web	serveur d'app	serveur SGDB
vue	logique métier	persistance des données

+++ *couplage faible*: code métier indépendant des périphs
enfichables => **archi hexagonale**

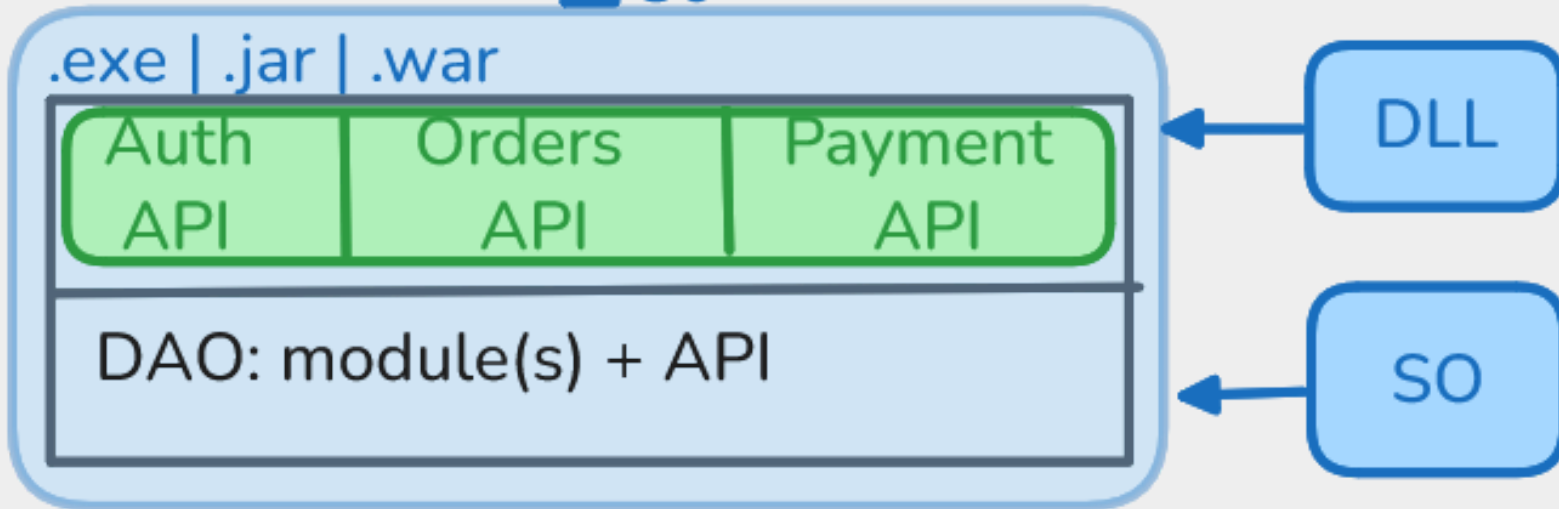
+++ séparation physique des responsabilités

--- *dépendance* au réseau et *latence* des appels distants

--- déploiement plus complexe

UI: module(s) + API
app mobile | SPA | legacy web client
http + Rest + Json | http + HTML

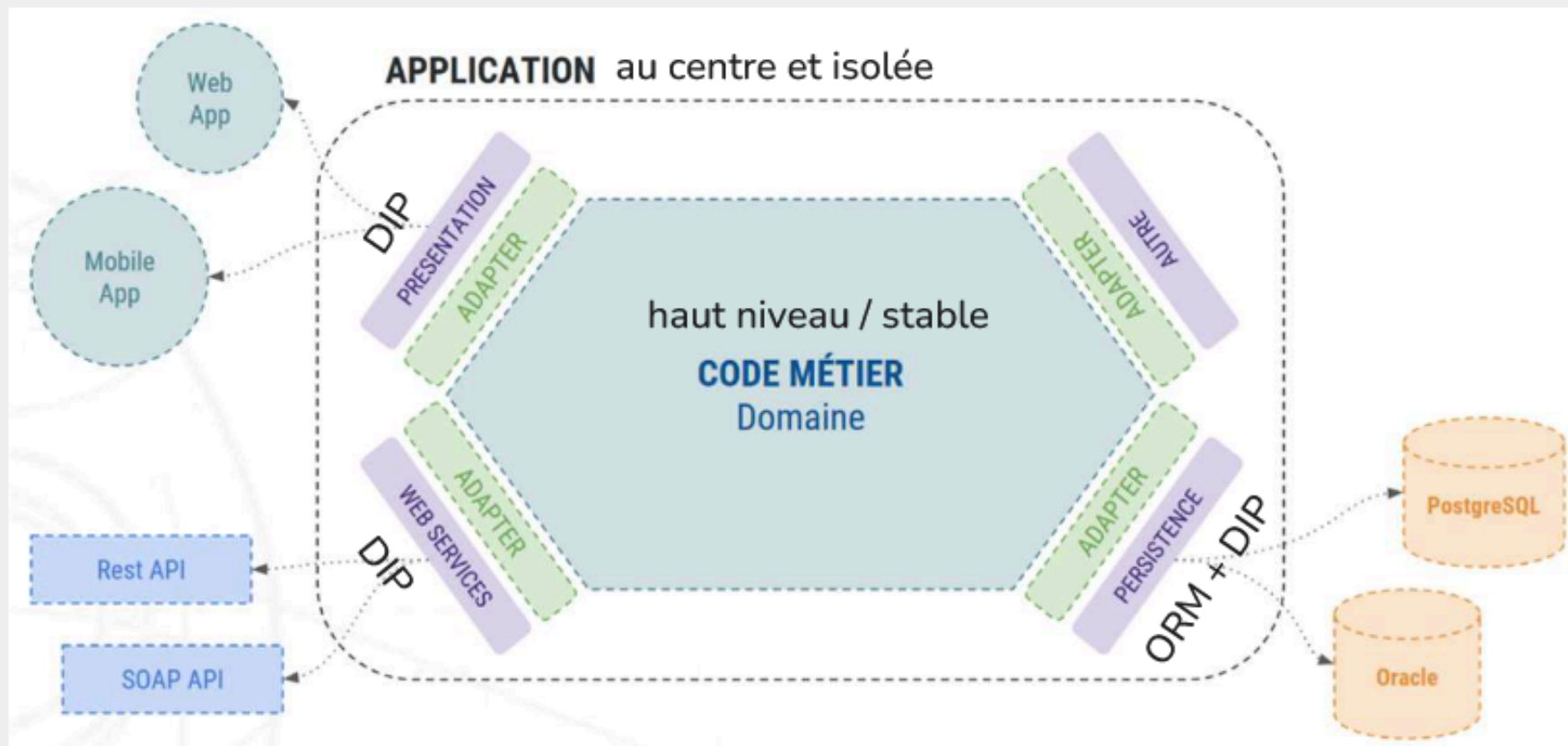
■ 80



tcp + psql ■ 5432

SQL | NoSQL | Fichier | block | objets
Data: module(s) + API

couches: architecture hexagonale



“ reformulation de l'archi propre pour les couches

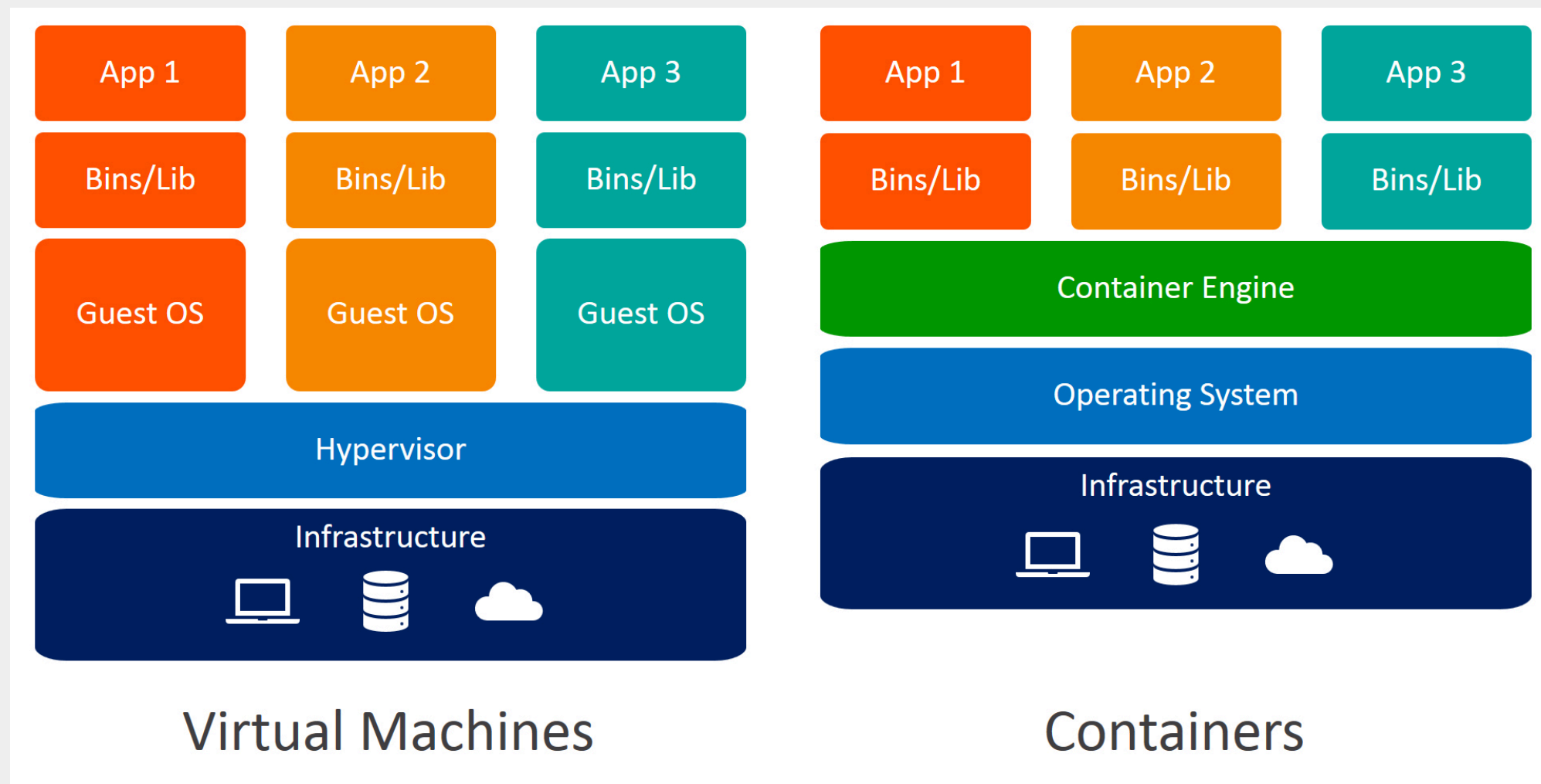
couches: unité d'exécution

- utilisation de services
 - isolation des processeurs
 - et communication à travers le réseau (sockets)

“ **on peut aussi isoler les processus**

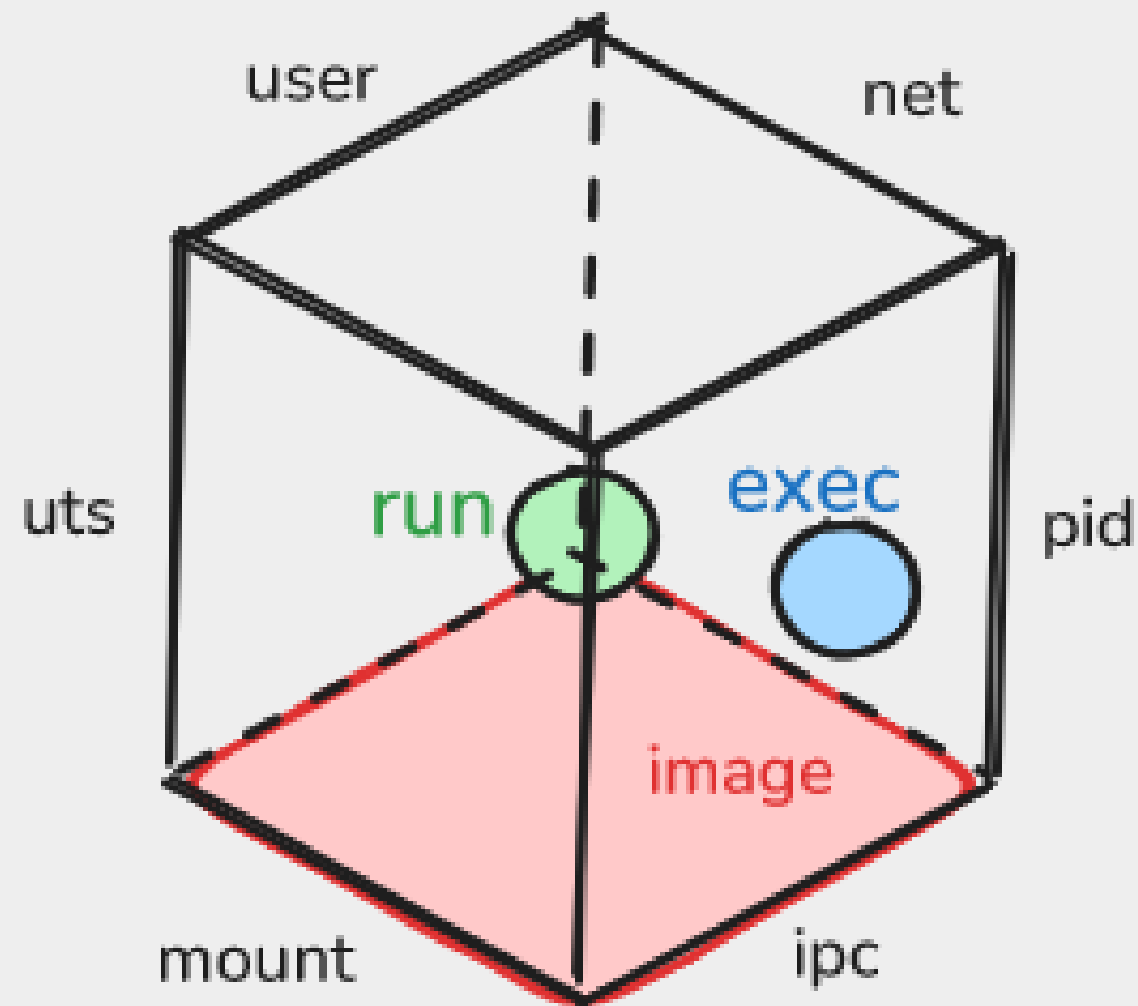
”

unité d'exécution: VM vs conteneur



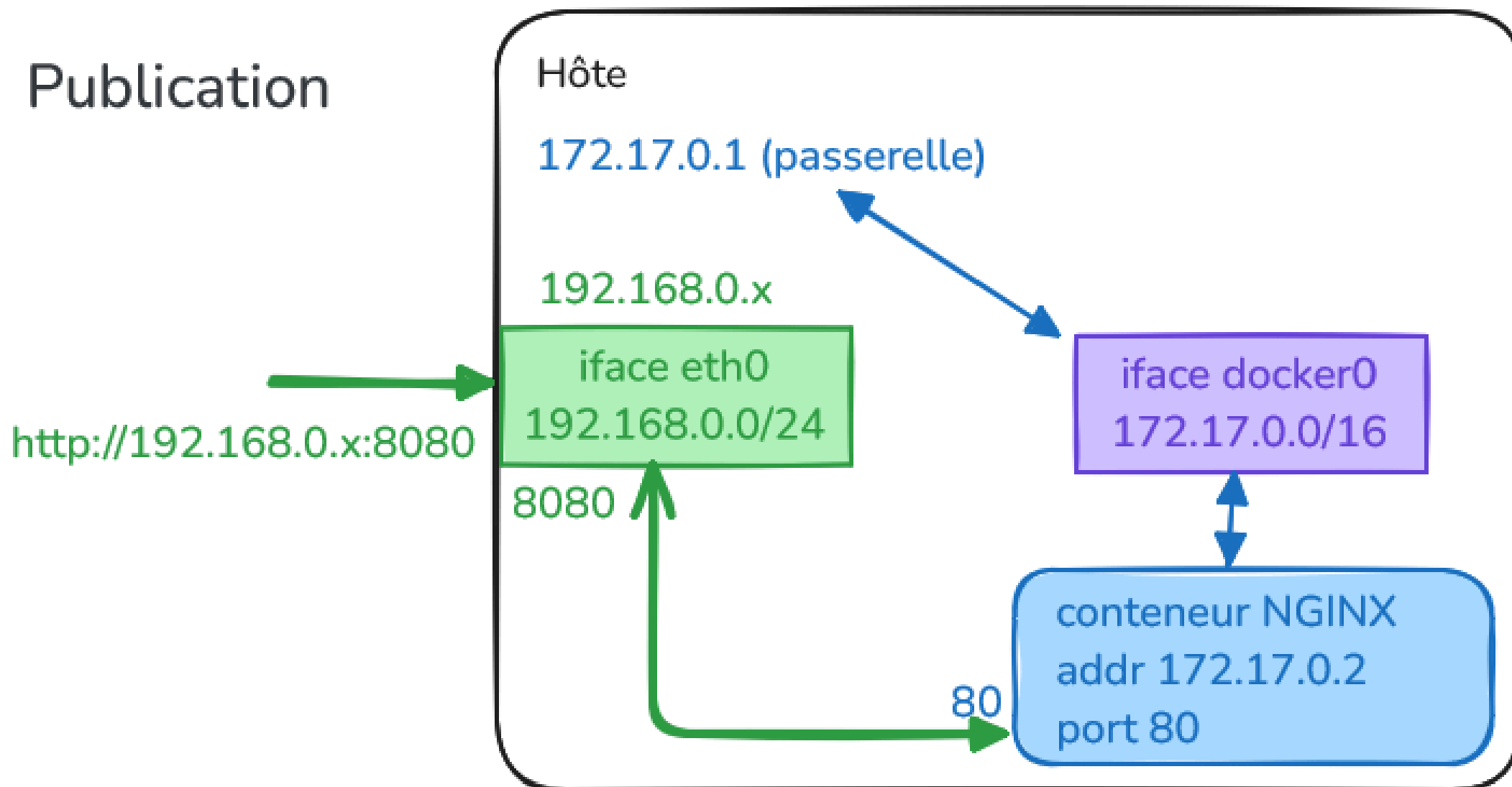
conteneur docker

- processus isolé
- par des **namespaces** du noyau linux
- assis sur un *ystème de fichier "image"* délimité
- communique sur le **réseau**



mise en réseau Docker

Publication



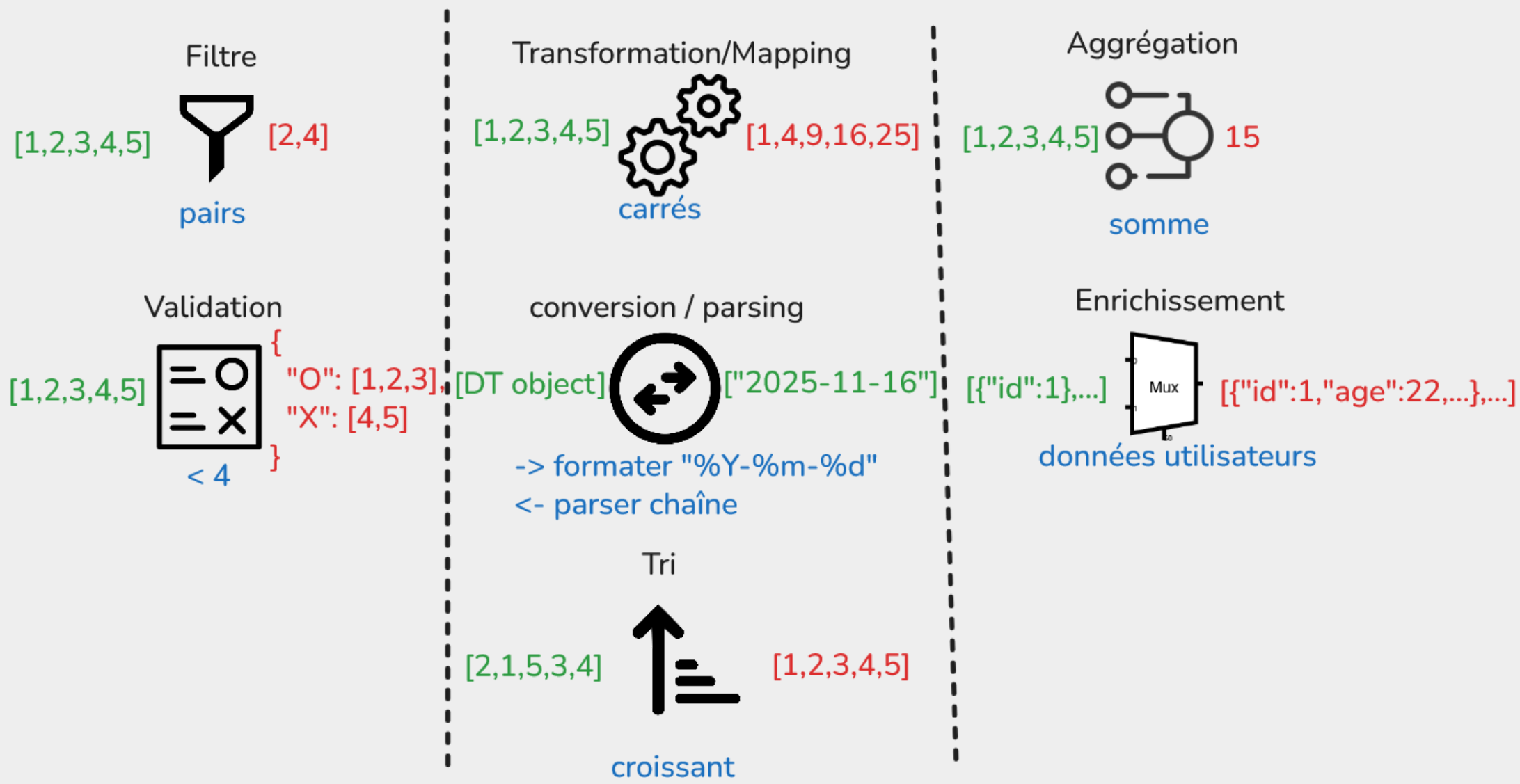
III.3 Flux de données

- Le système est décomposé en *composants de traitement*
 - connectés par des *canaux de flux de données*
 - stratégie « **Pipes & Filters** »

```
cat file.txt | grep "error" | sort | uniq -c | head -10
  ↑         ↑         ↑         ↑         ↑
source    filter    sort    count    limit
```

- lien avec la **programmation fonctionnelle**
 - composition de *fonctions pures*
 - *immutabilité*

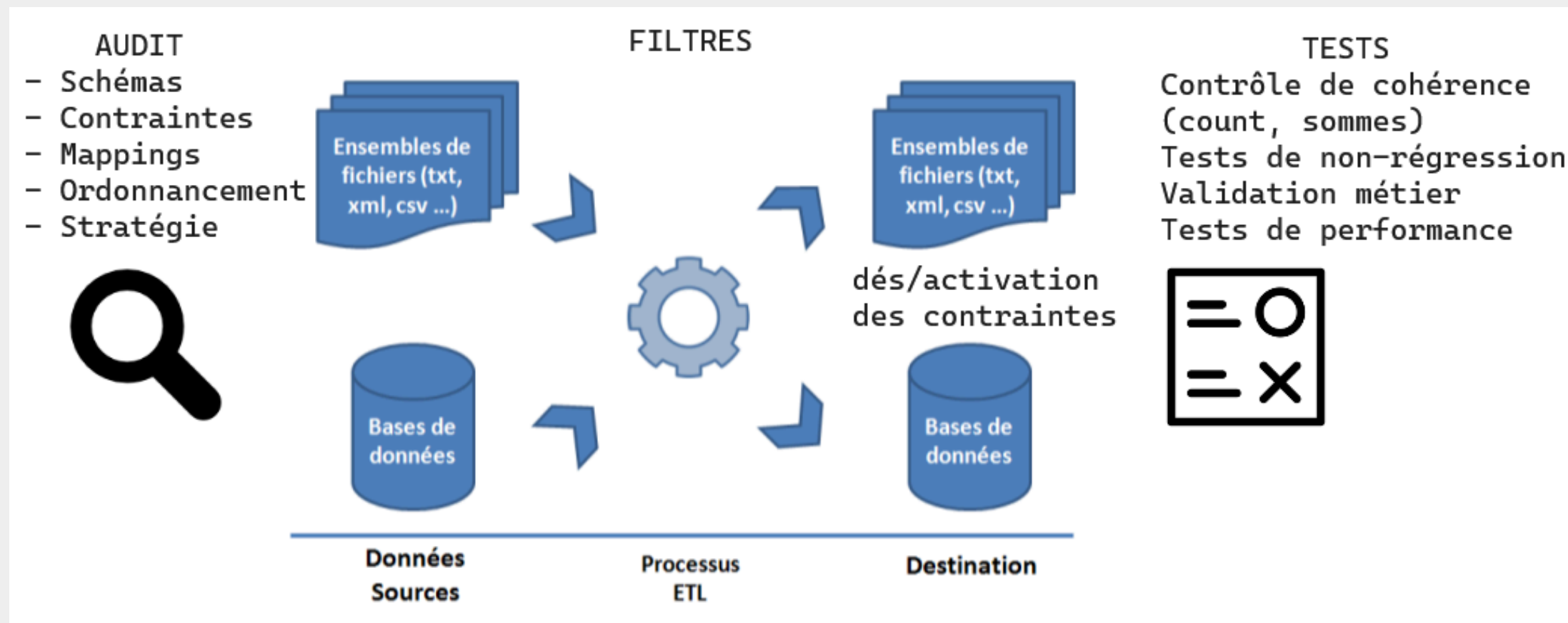
Flux de données: « Filtres » généraux



Flux de données: usages remarquables

- ETL: **E**xtract, **T**ransform, **L**oad
 - *Transformation entre plusieurs sources de données*
- ELT: **E**xtract, **L**oad, **T**ransform
 - *multiples Analyses/Transformations après Chargement*
- CI/CD: **C**ontinuous **I**ntegration / **C**ontinuous **D**elivery
 - *Build* (parse), *Test* (validate), *Deploy* (enrich)
- Exploitation des Logs: *Parse* → *Enrich* → *Index* (sort)

ETL: migration de modèles de données



progressive (par modules) et/ou *incrémentale* (par batches)
big bang (tout synchrone d'un seul coup) / *parallèle* (verrous)

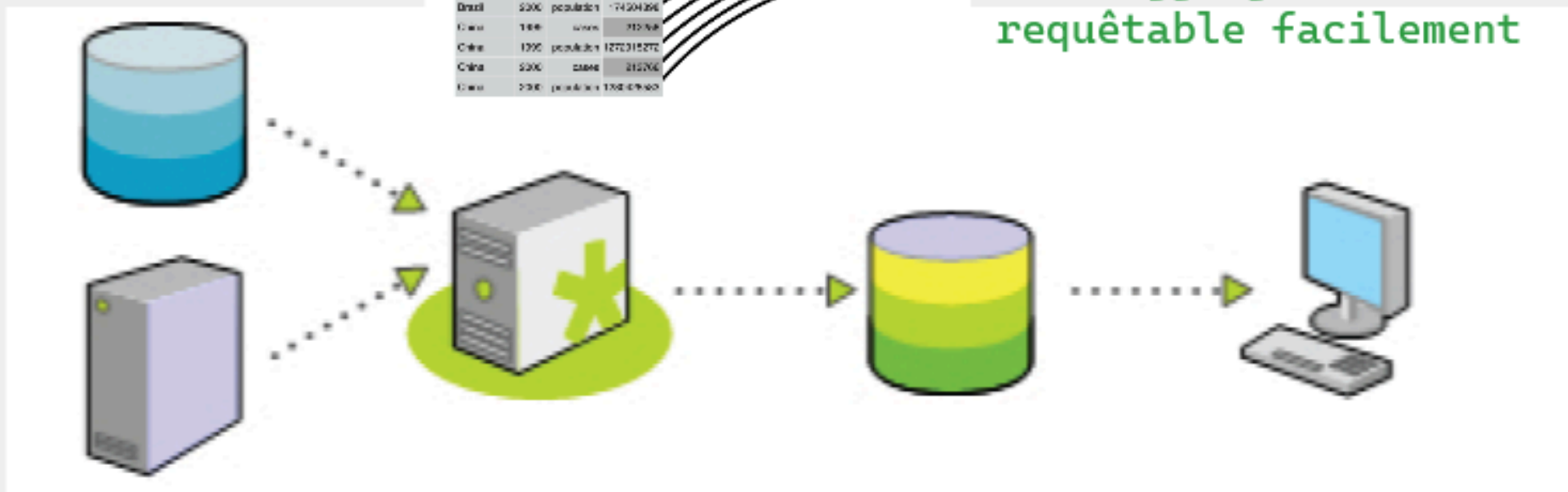
ETL: intégration décisionnelle

Modèle normalisé
grand nb de
tables longues
avec bcp d'indexes

ETL

country	year	key	value
Algerie	1999	maison	225
Algerie	1999	population	16207371
Algerie	2000	maison	2488
Algerie	2000	population	20505362
Brazil	1999	maison	37250
Brazil	1999	population	17038640
Brazil	2000	maison	60468
Brazil	2000	population	174204266
China	1999	maison	21256
China	1999	population	1270315272
China	2000	maison	212766
China	2000	population	1280438885

Modèle dénormalisé
petit nb de
tables larges et jointes
peu d'indexes
avec agrégats
requêtable facilement



Bases Transactionnelles

DataWarehouse
schema étoile ou flacon

Business Intelligence

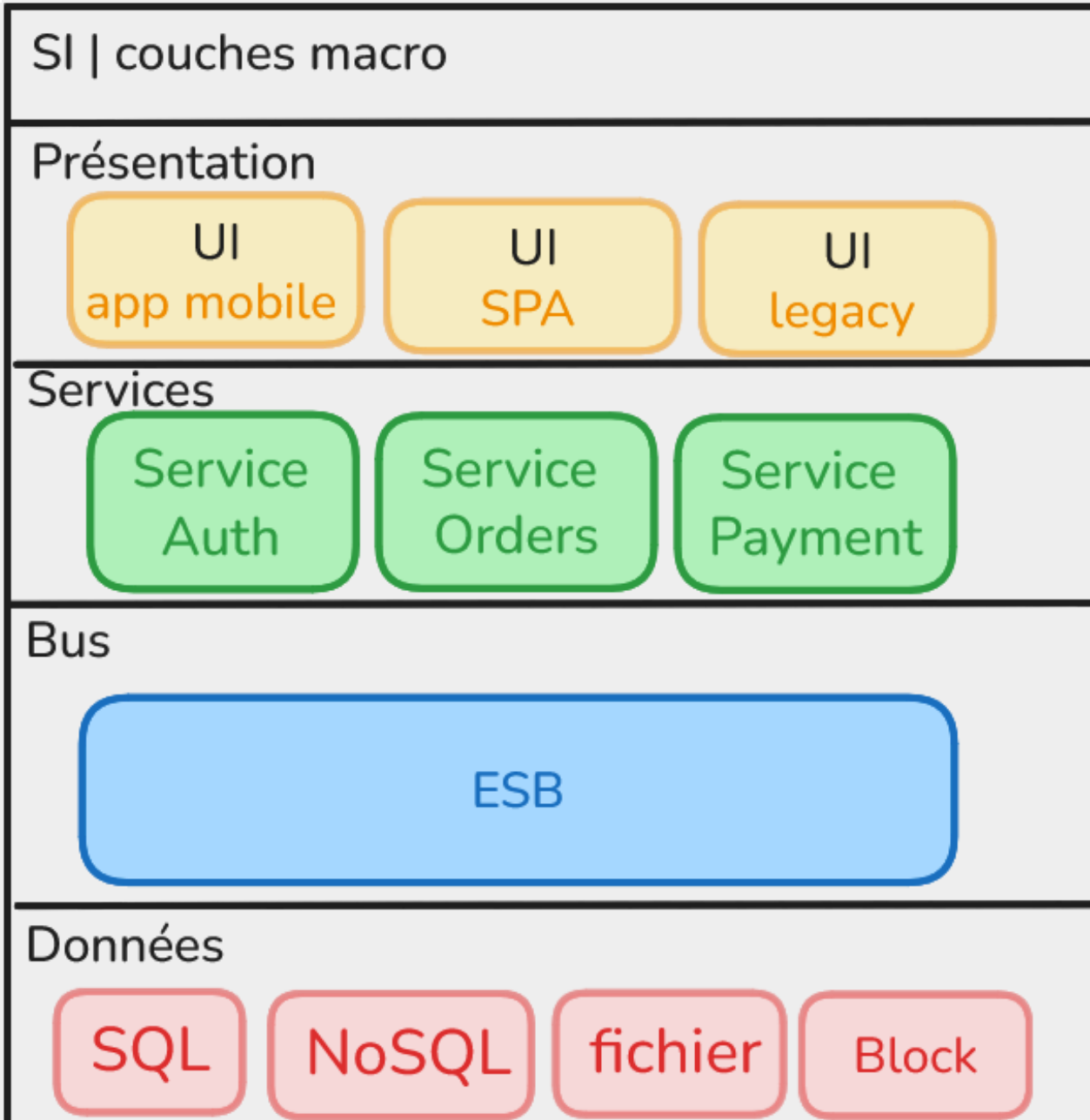
III.4 Architecture distribuée

- *stricto sensu*: système réparti sur plusieurs nœuds physiques
=> client-serveur et n-tiers sont distribués
- architectures distribuées sans couches:
 - **Orientée Services (SOA)**
 - **Microservices**
 - **Orientée Agents / Service Mesh**

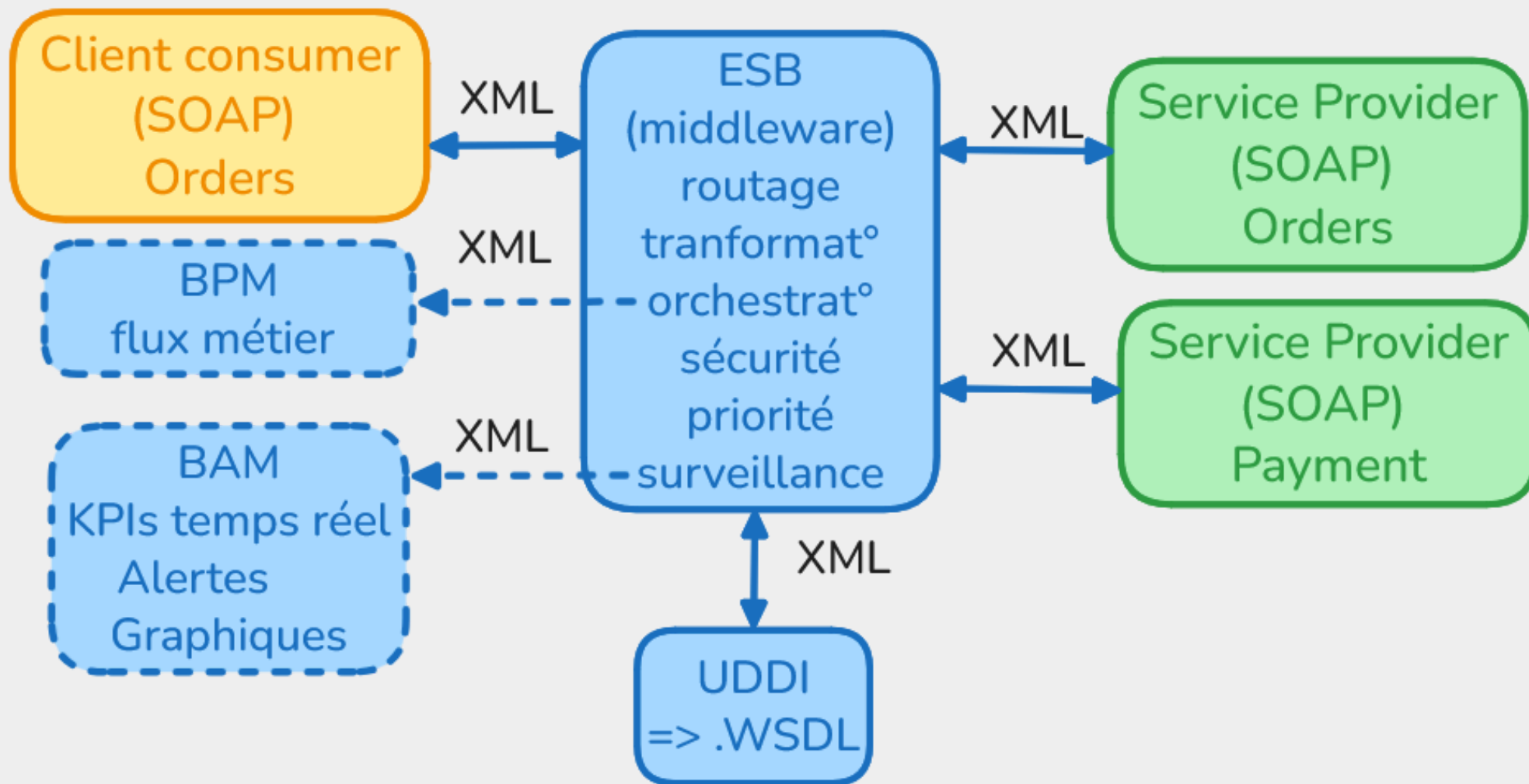
SOA

- architecture installée sur le SI global d'un **réseau d'entreprise**
- dont les sous-systèmes représentent les *services business*
- ces sous-systèmes haut niveau sont des **services autonomes**
 - intégrés et communiquant via un **bus de services d'entreprise (ESB)**.

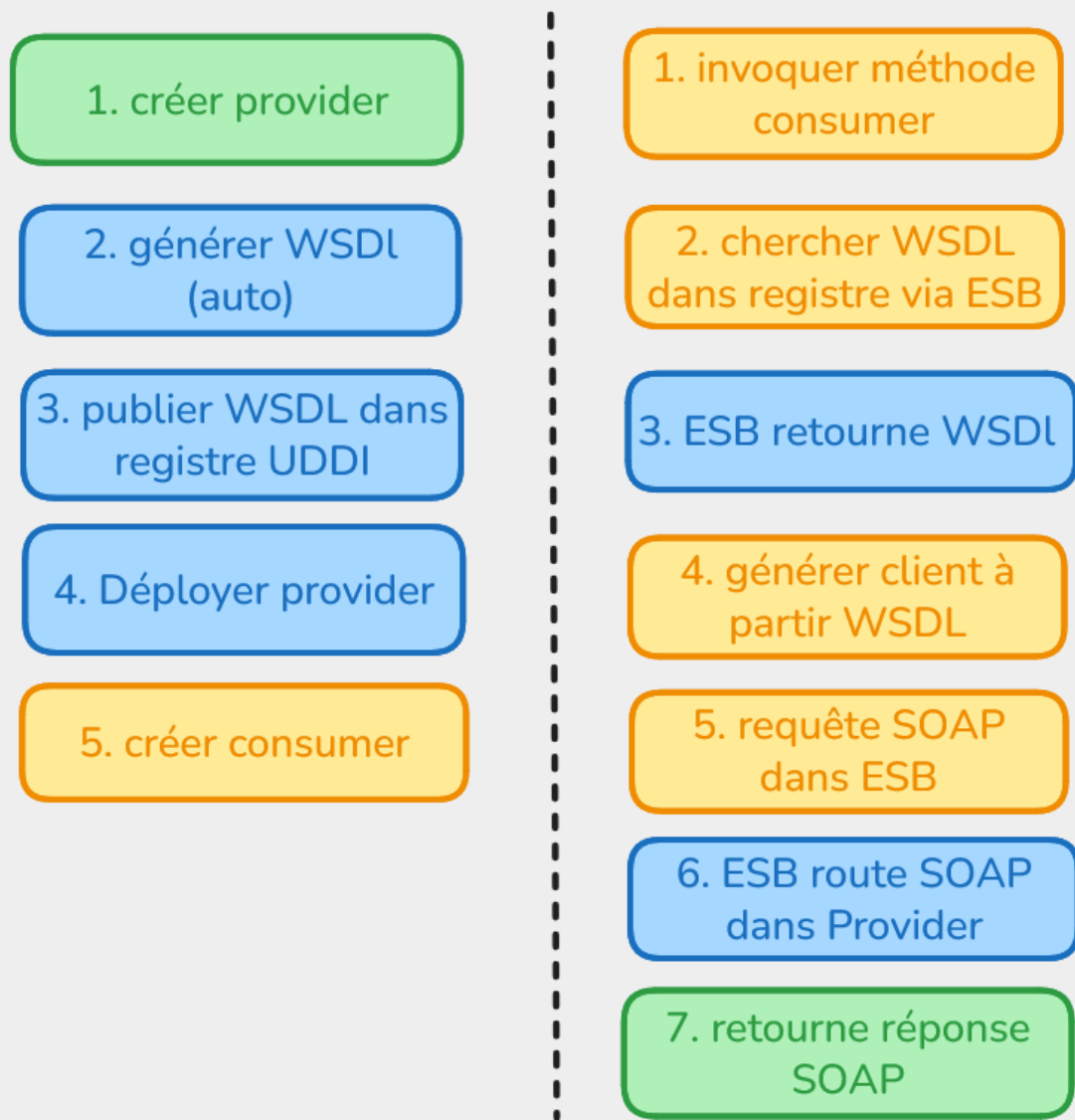
“ approche EAI: **E**ntreprise **A**pplication **I**ntegration ”



SOA: composants techniques



SOA: flux WSDL / SOAP



SOA: contrats d'interface WSDL

<u>contrat WSDL</u>
TYPES : Contrat des DONNÉES corps de requêtes / réponses (XSD)
MESSAGES: Contrat des Messages Enveloppes requêtes / réponses (XML)
TYPE PORTS: Contrat des OPERATIONS Interfaces des méthodes providers (XML)
BINDS: Contrat des DETAILS TECH. entêtes (XML)
SERVICES: Localisation du provider (XML)

SOA: avantages / inconvénients

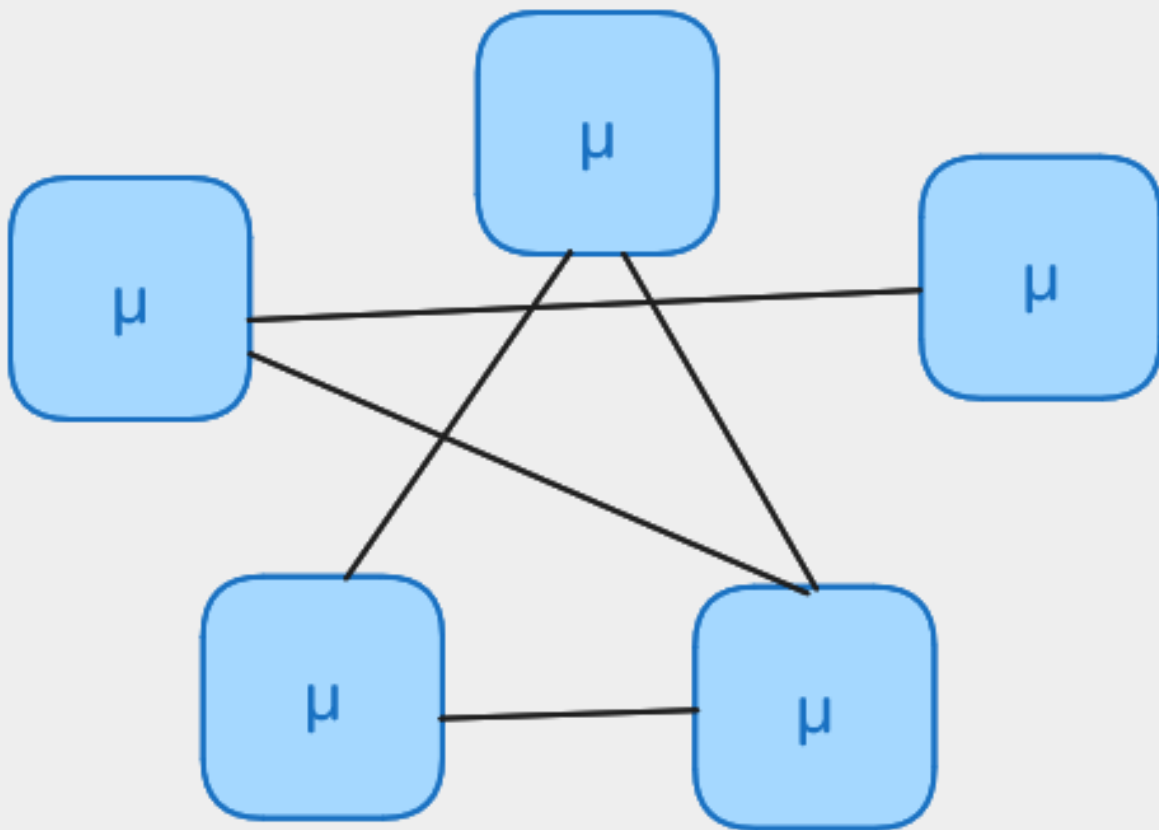
- **+++** : couplage inexistant entre services
- **+++** : réutilisation des services pour nombreux cas d'usage
- **---** : lourdeur des messages SOAP / WSDL
- **---** : pas de client léger => proxy backend pour les consumers
- **---** : complexité de mise en œuvre d'un ESB
- **---** : l'ESB est un gros SPOF

Microservices

stricto sensu:

- évolution de la *SOA*
- chaque **μservice** est une **application légère autonome**
 - déployée indépendamment
 - communiquant via des *APIs légères* (REST / GraphQL)
- chaque service implémente un **cas d'usage métier spécifique**
 - et gère ses *propres données*

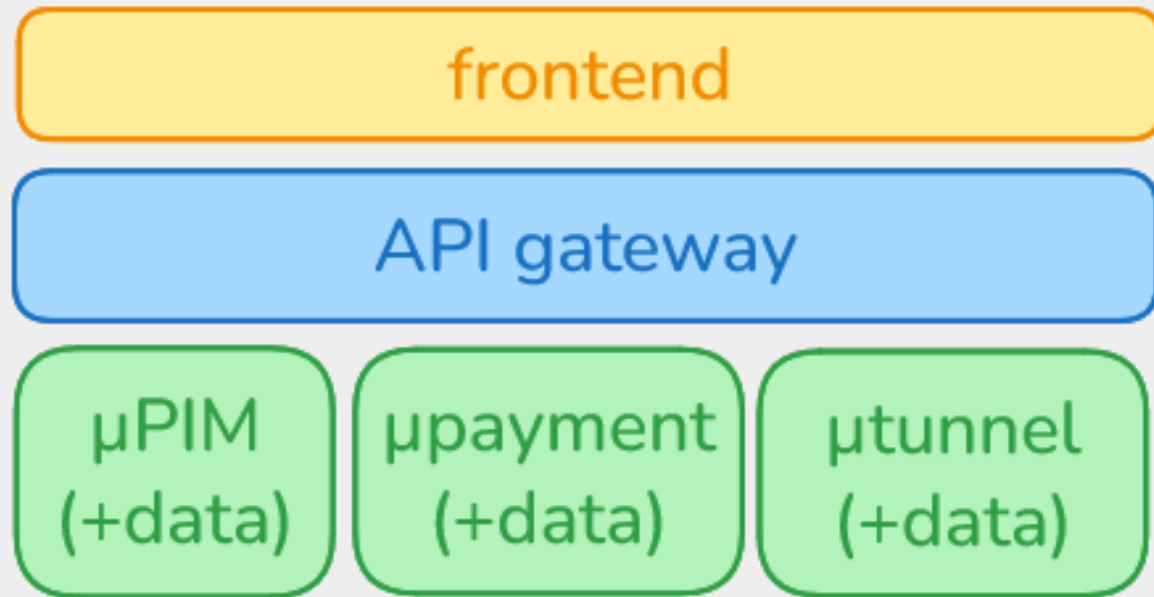
Microservices: schéma général



“ **peu praticable sans orchestration**

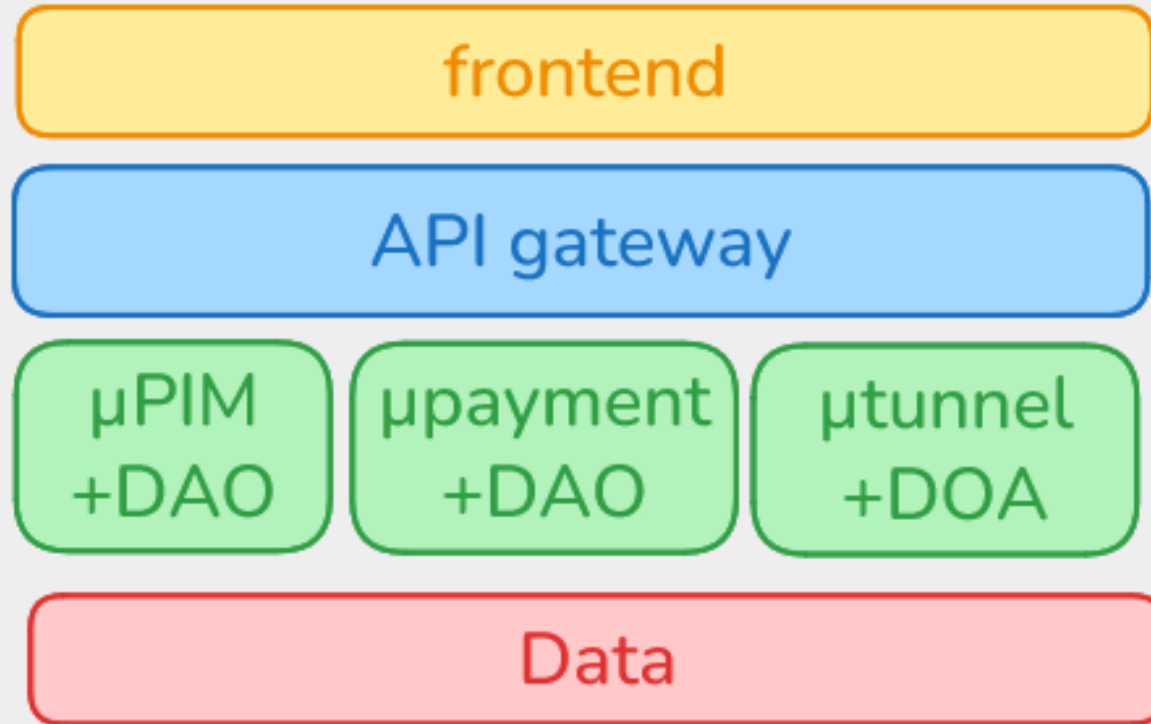
”

Microservices: dans une archi "3-tiers"

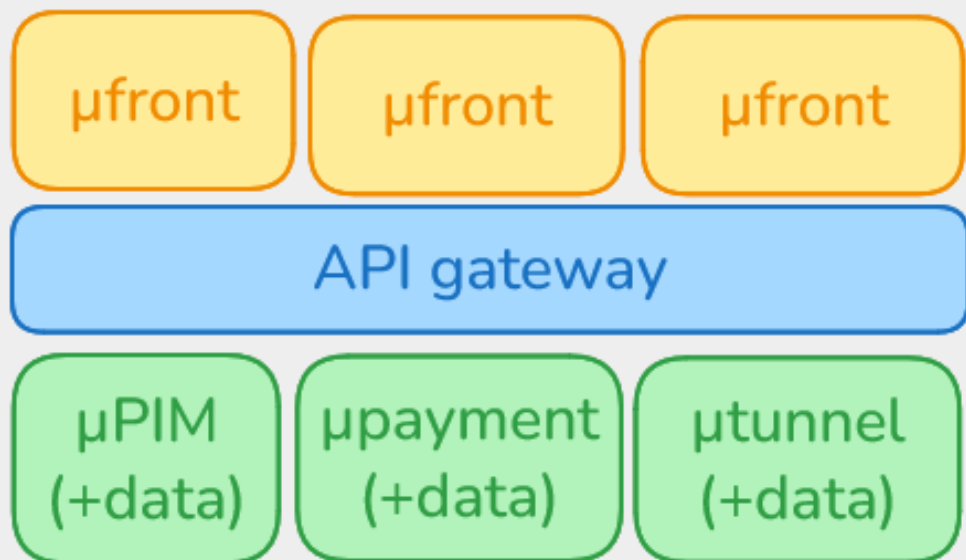


- μservices comme briques ERP
- la **passerelle API** route les requêtes aux bons services

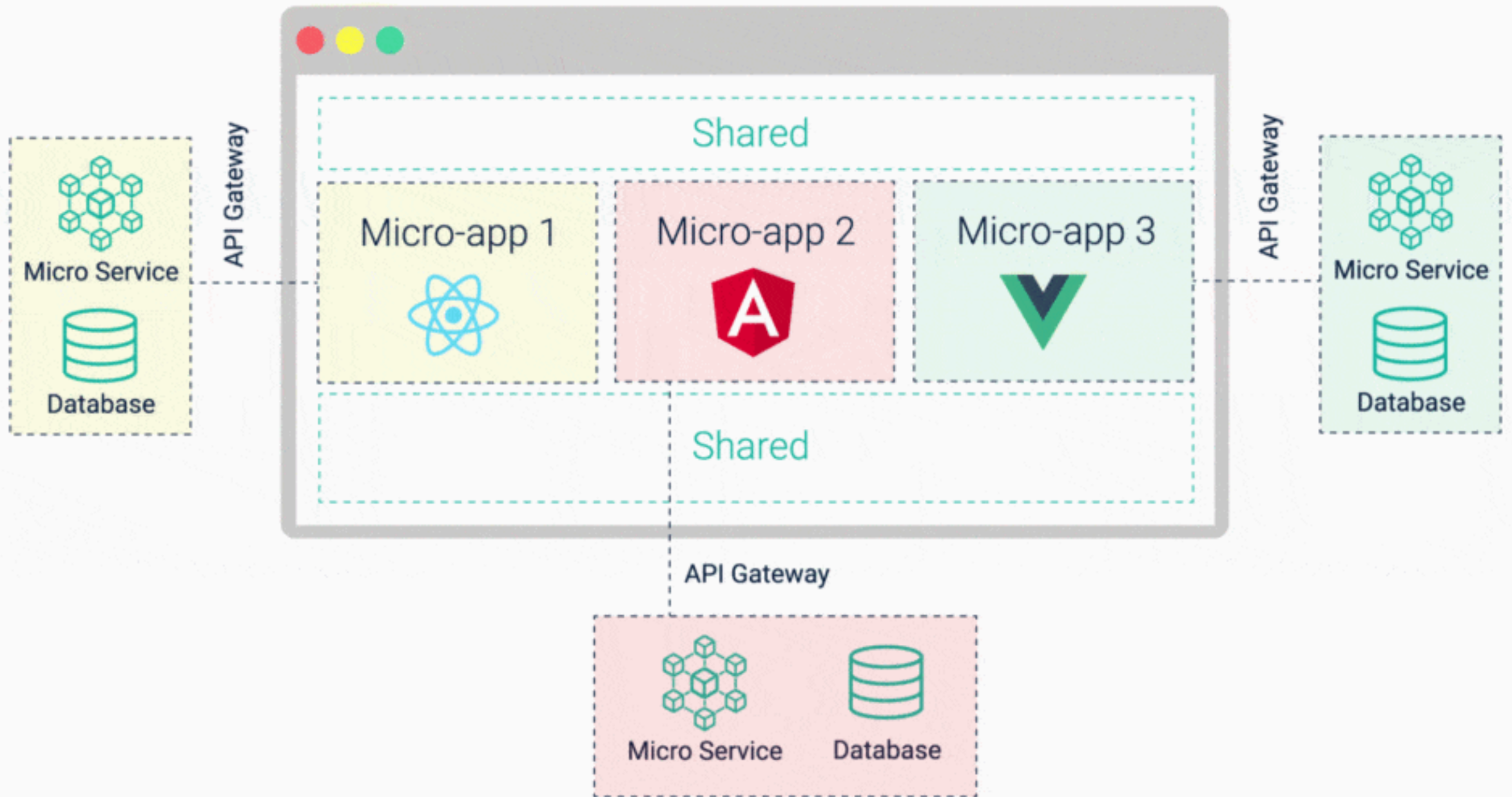
Microservices: variation



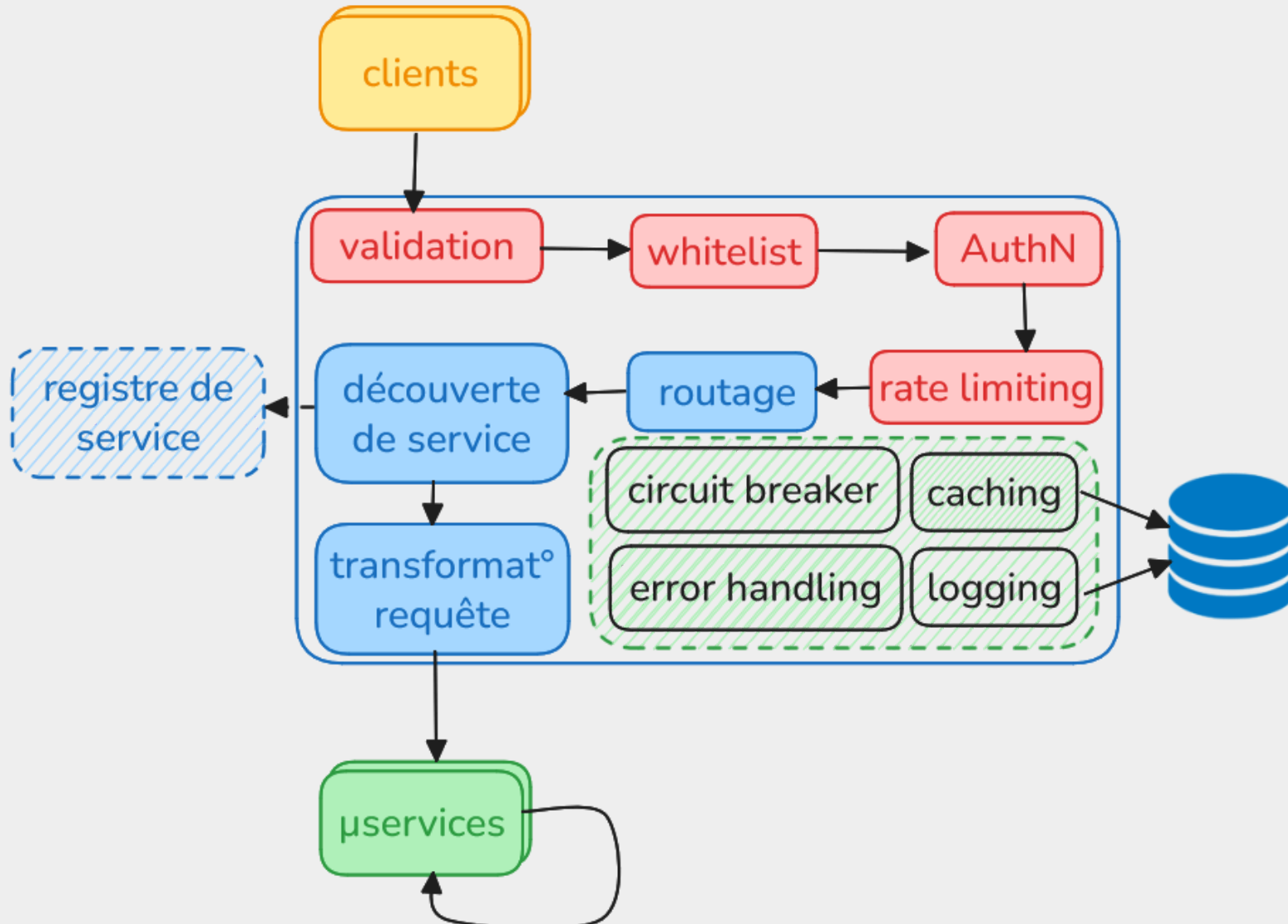
Microservices: micro frontends



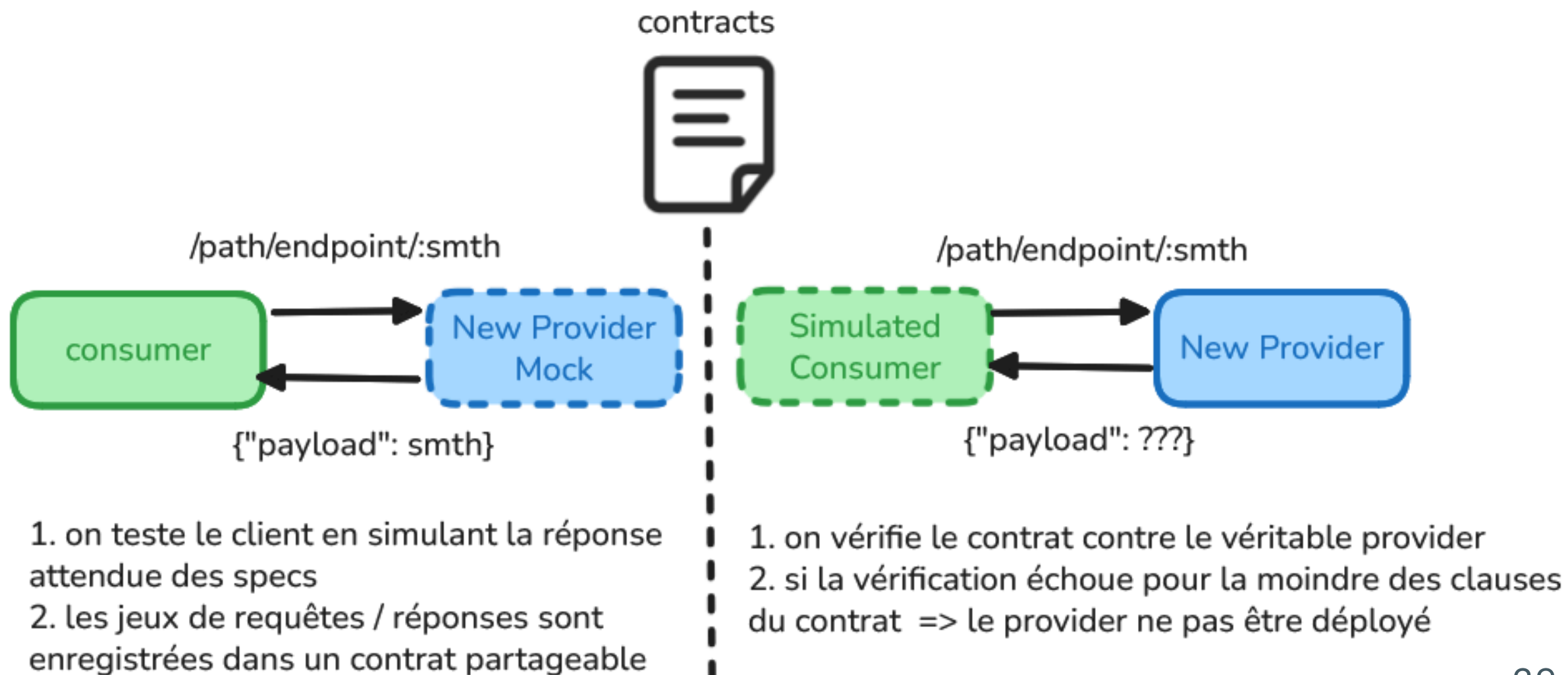
- un micro front est un composant UI autonome
- le *gabarit* est souvent une *SPA* (Single Page Application)
- *+++* : déploiement indépendant des équipes UI / UX
- *---* : complexité de gestion des états globaux



Microservices: gateway API



Microservices: respect des contrats



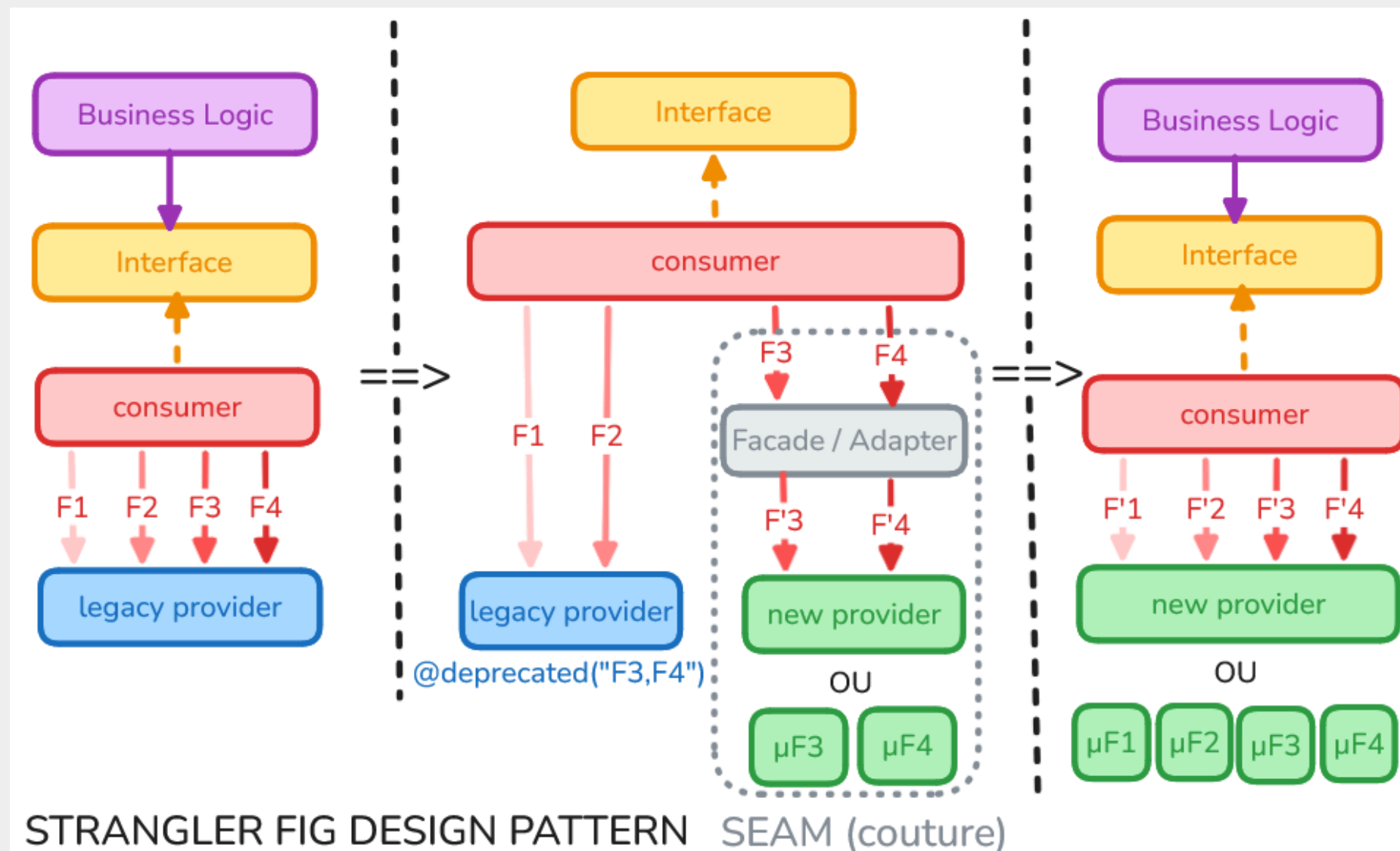
Microservices - gateway API: fonctionnalités

- la passerelle est analogue à un **serveur proxy inverse**
 - *routage* des requêtes aux services appropriés
 - *agrégation* des réponses de plusieurs services
- les possibilités de transformation sont *plus réduites*
 - url rewriting
 - conversion de protocoles (ex: REST → gRPC)
- les fonctionnalités création / stockages
 - doivent utiliser des services **stateful** délégués
 - pour *préserver la scalabilité de la passerelle* **stateless**

Microservices: pourquoi les utiliser ?

- Avantages / Inconvénients
 - **+++** : déploiement indépendant des services
 - **+++** : scalabilité horizontale fine des services critiques
 - **---** : complexité de gestion des communications inter-services
- quand le *monolithe devient trop gros / lent / complexe*
- quand les équipes de dev sont *nombreuses et spécialisées*
- “ la loi de Conway (1967)
L'architecture d'un système reflète la structure de communication de l'organisation qui le conçoit

Monolithic modulaire ==> Microservices



Archi orientée agents

- chaque **agent** est un **composant logiciel autonome**
 - qui possède une *intelligence* pour
 - percevoir son environnement
 - prendre des décisions
 - agir de façon autonome

“ l'agent est un véritable **microservice intelligent**

”

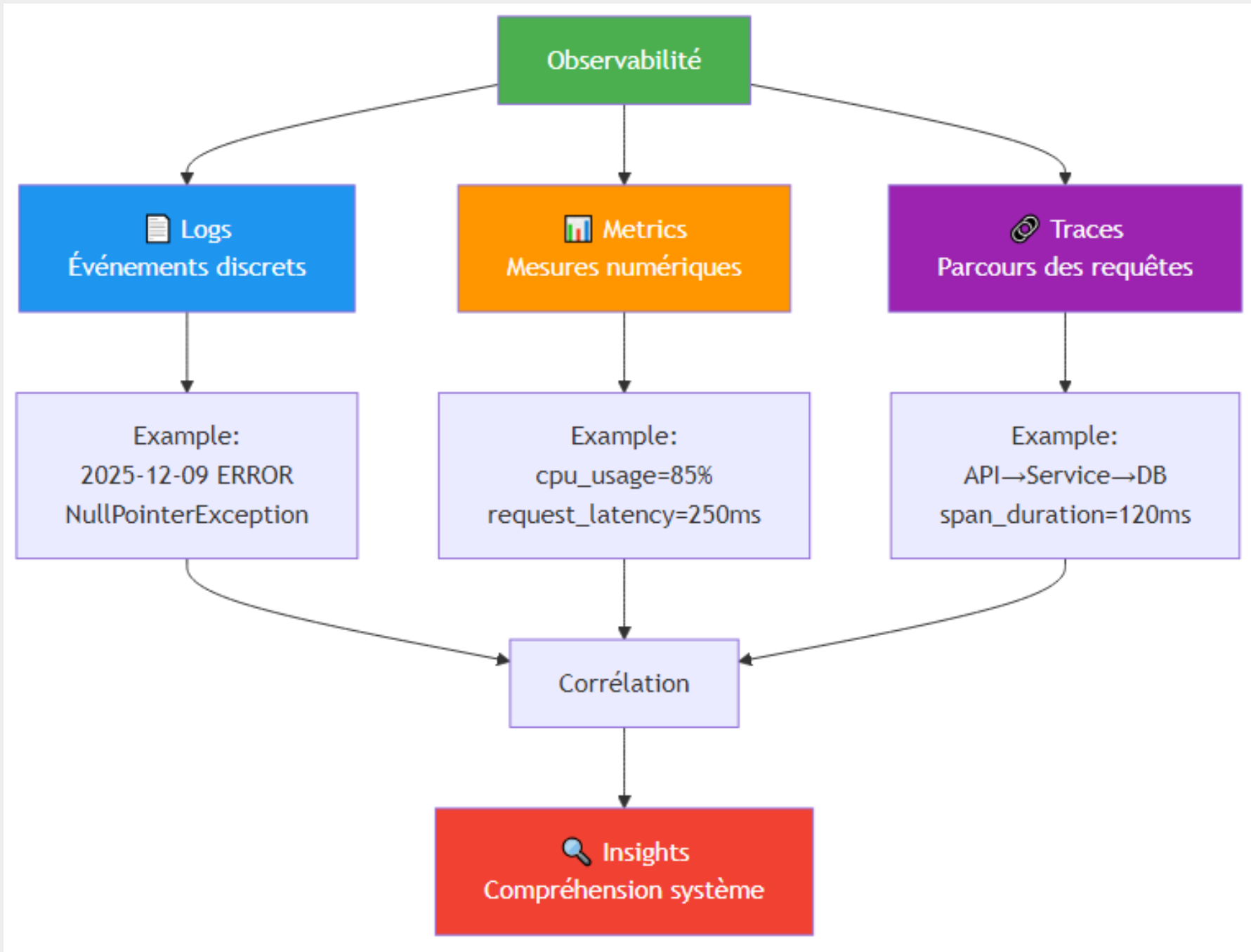
Usage de l'Archi agents: Observabilité

- **monitoring:**

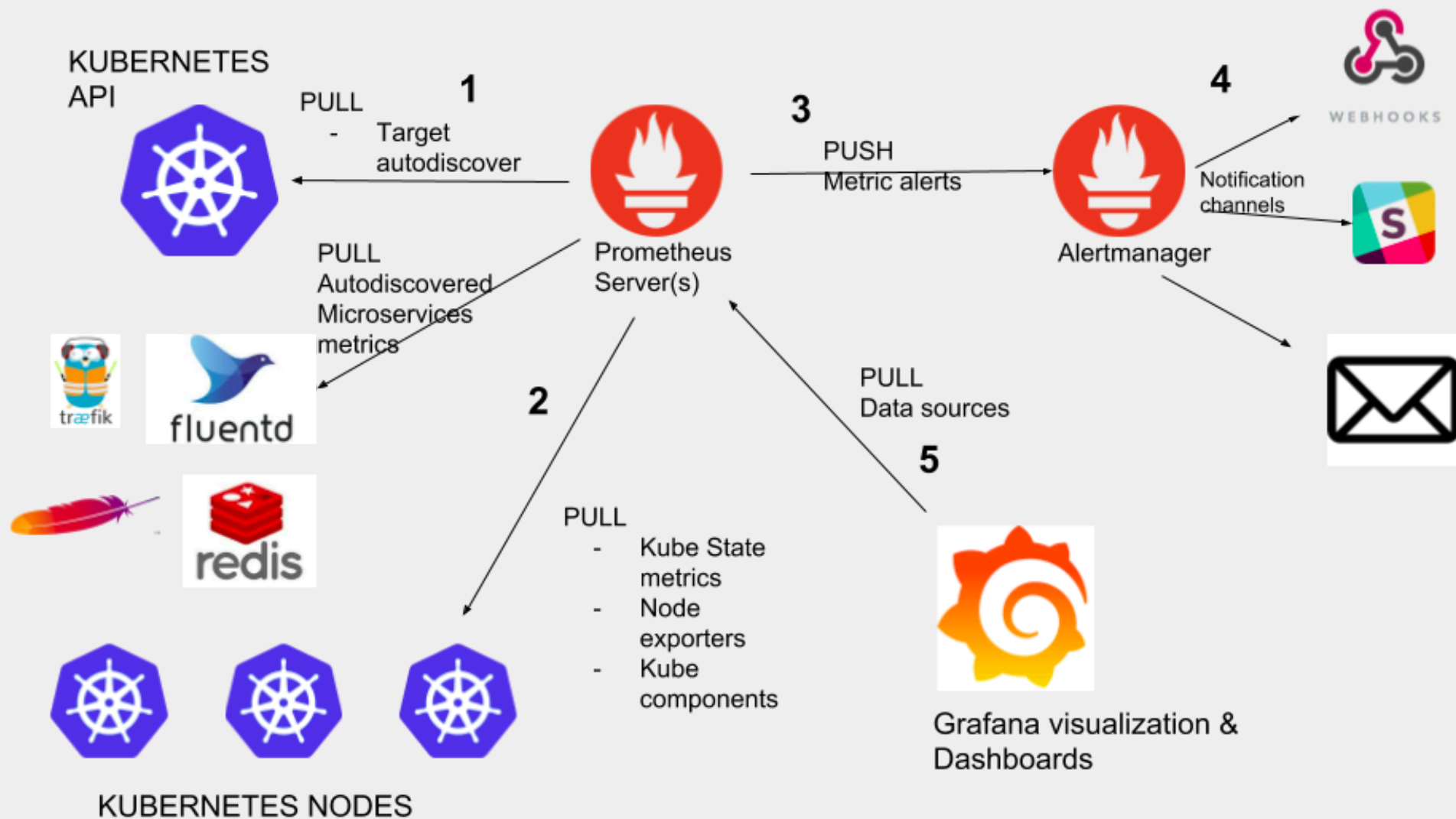
“ surveillance continue et automatisée d'un système informatique pour *vérifier qu'il fonctionne correctement* et détecter les anomalies ==> **voyants du tableau de bord (t°, vitesse)** ”

- **Observabilité:**

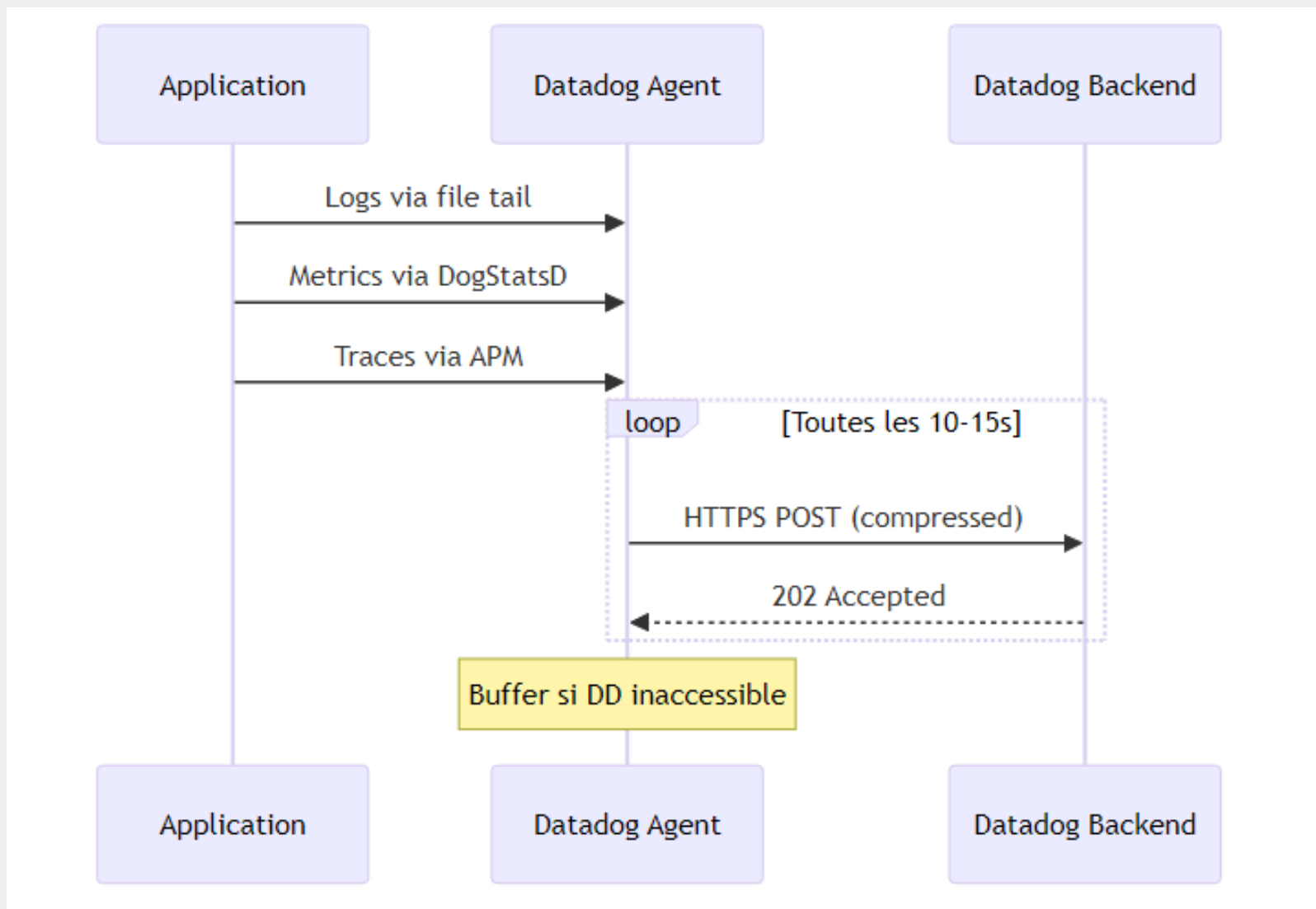
“ *capacité à comprendre l'état interne* d'un système en examinant ses sorties externes (logs, métriques, traces). C'est une propriété du système qui permet de diagnostiquer des problèmes sans avoir à le modifier ==> **Boîte noire + tous les capteurs** ”



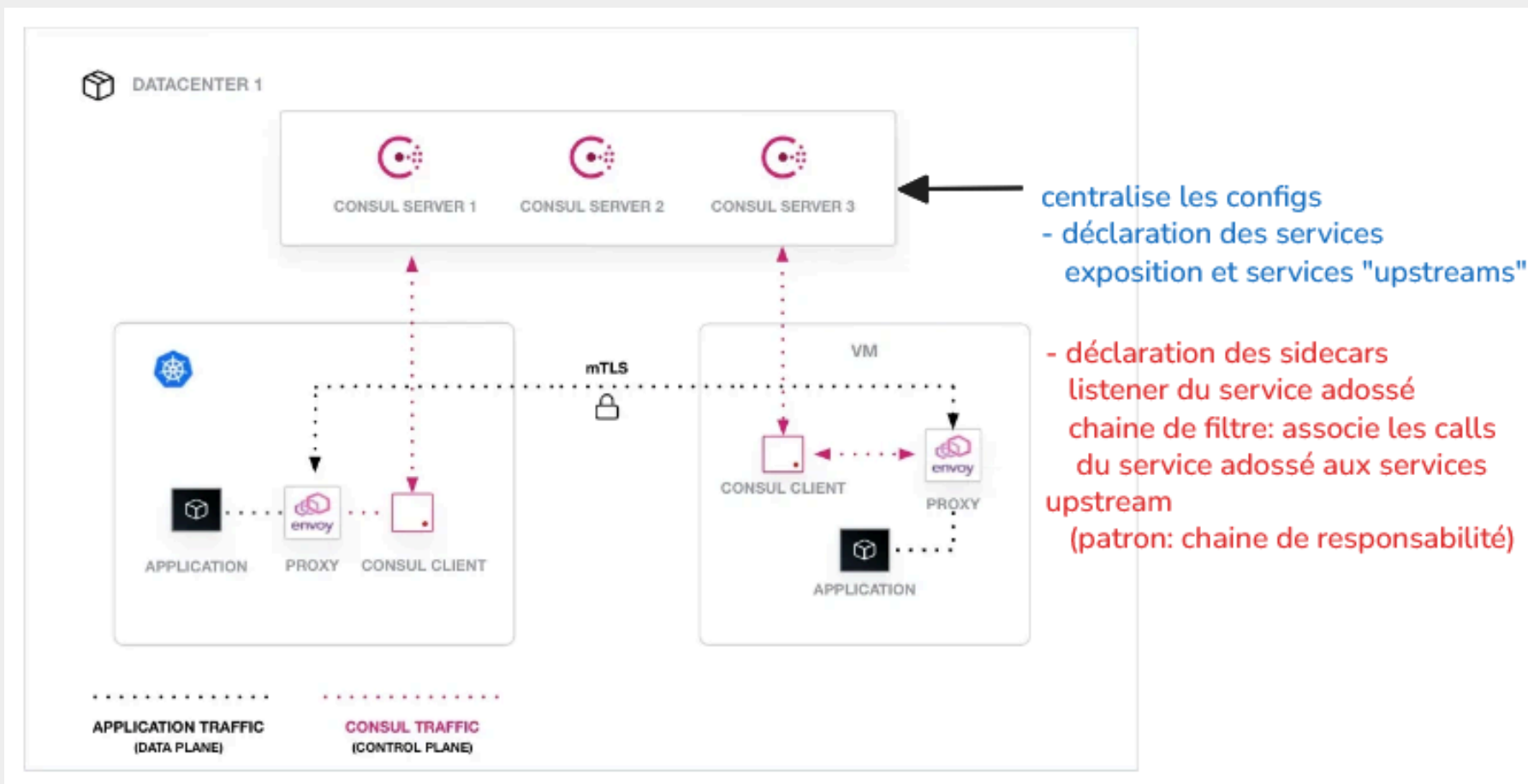
Archi agents: exemple monitoring



Archi agents: exemple observabilité



Usage de l'Archi agents: Service Mesh

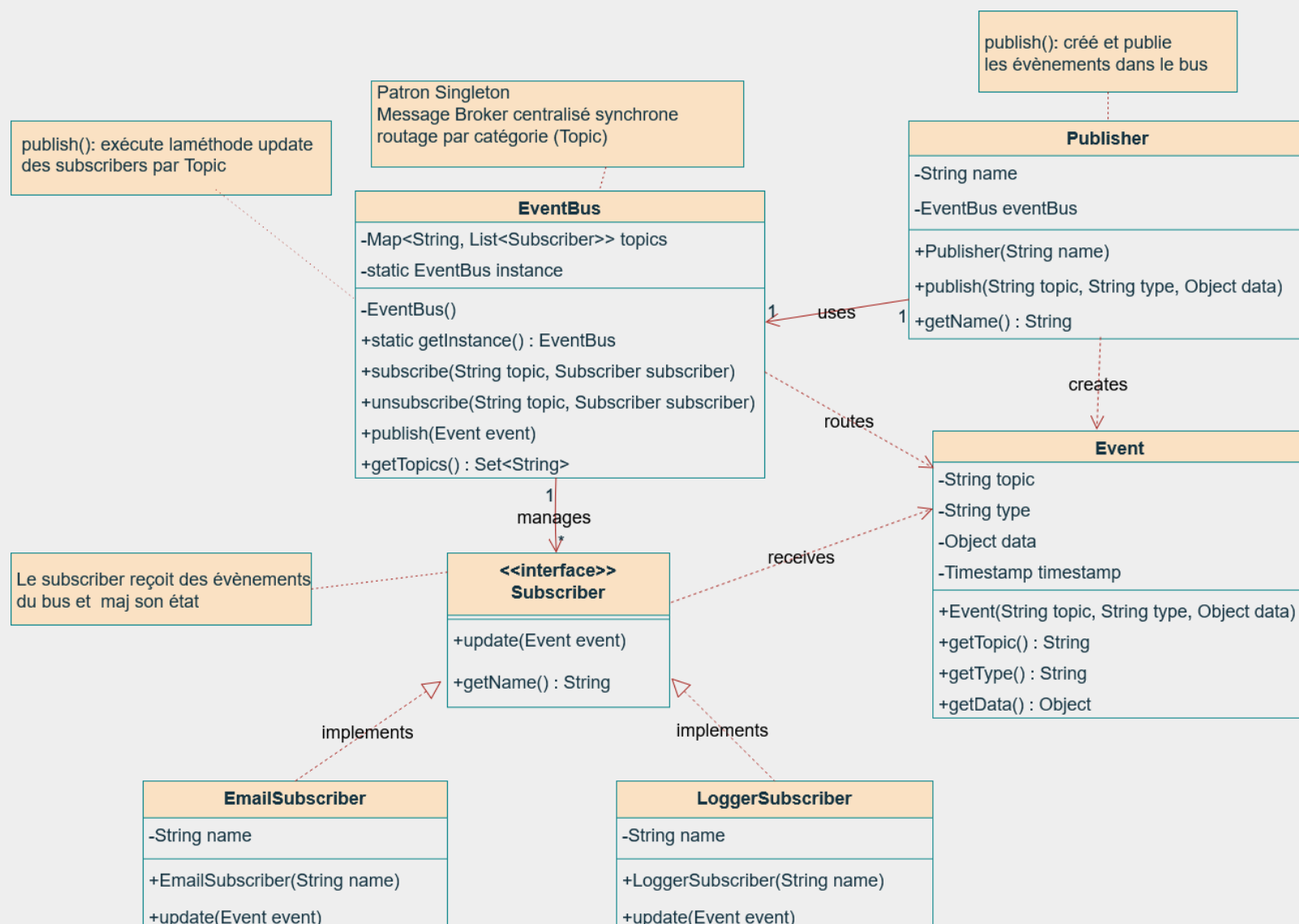


“ les proxies "side car" forment une passerelle API distribuée !

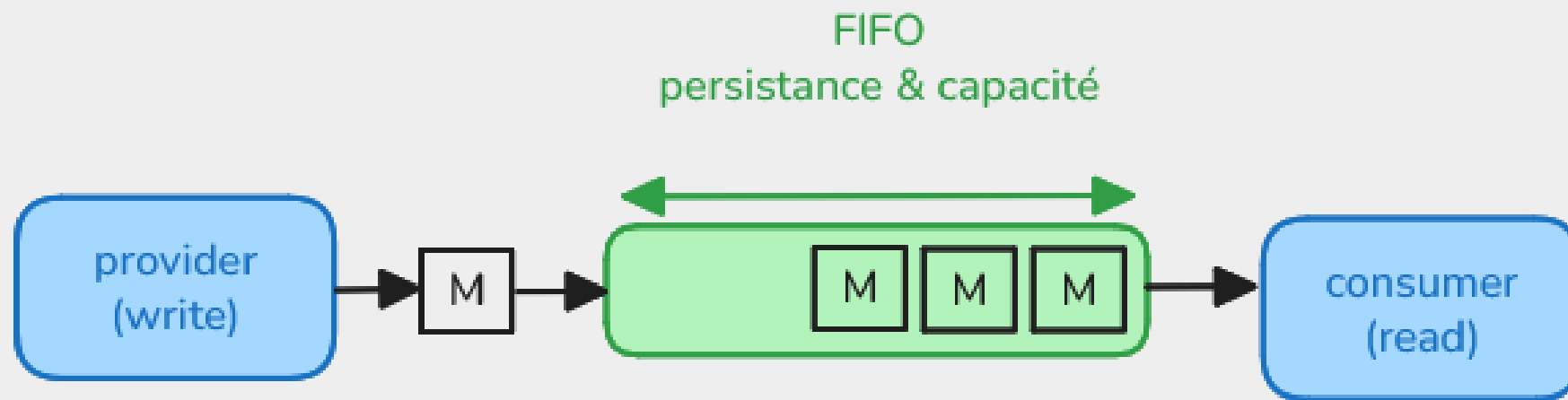
III.5 Architecture orientée évènements

- **Event-Driven Architecture (EDA)**
- les composants communiquent via des **évènements asynchrone**
 - produits par des *producteurs d'évènements*
 - consommés par des *consommateurs d'évènements*
 - les évènements sont des *"faits" => immutables*

EDA: EventBus synchrone



EDA: Message Oriented Middleware (MOM)



- provider , FIFO, et consumer sont des *services*
- les écritures / lectures sont **asynchrones**
=> Pas d'attente de réponse, pas de blocage
- grâce à une *file de messages persistante*

III.6 Architecture centrée sur les données

“ Place les données au centre du système, où plusieurs composants/services lisent et écrivent dans un **référentiel de données partagé**. ”

- ce référentiel représente une Source de Vérité unique (des *données critiques*)

SSOT

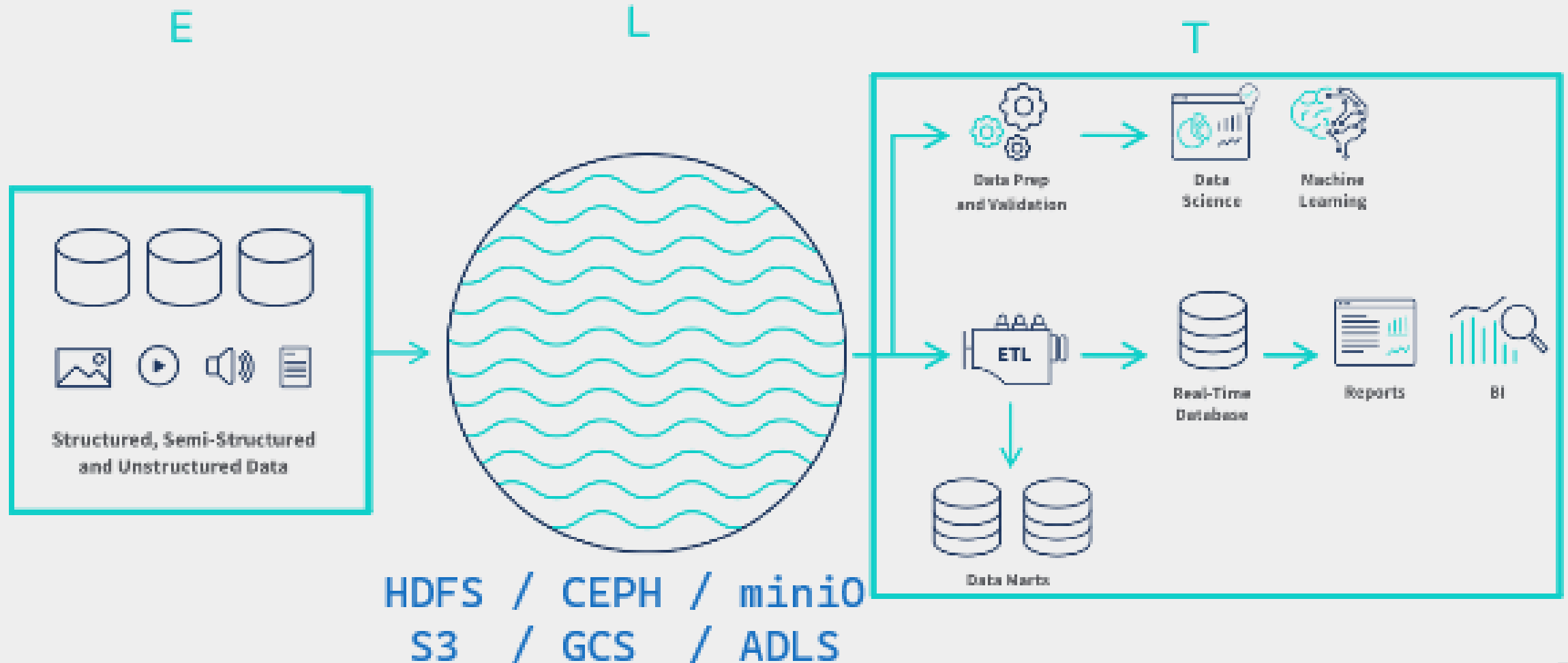
“ **Une donnée = Un seul emplacement de référence** ”

- peut être
 - une base de données / stockage en direct non distribuée
 - un *service* vers le précédent
 - *Data lake* en data engineering
 - *Dépôt de code source* de référence (ex: GitHub/Gitlab)
 - un *event store* en architecture orientée évènements

Data Lake

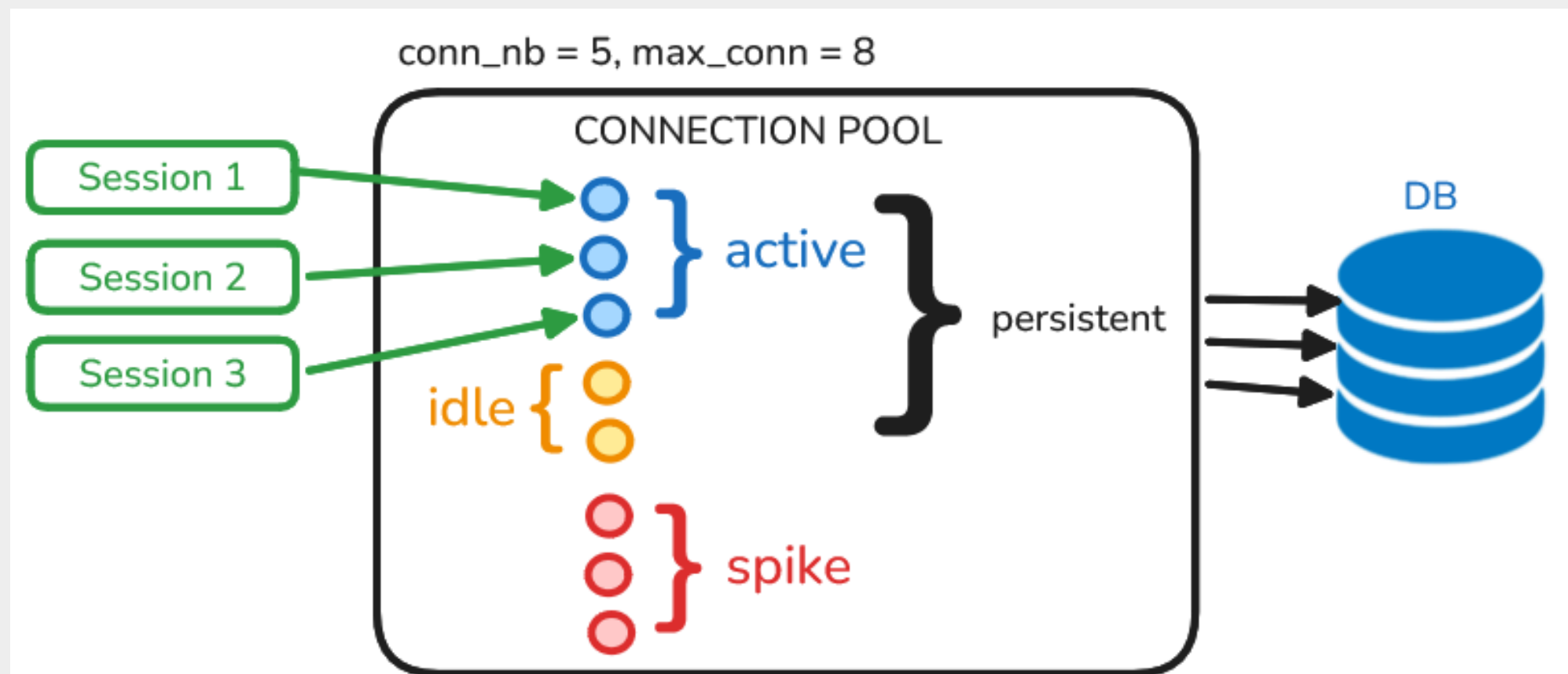
- **données brutes** stockées dans leur *format natif avec un ELT*
- ingestion de données provenant de *sources diverses*
- **source unique** pour analyses / machine learning / dénormalisation (data warehouse)
- **Schema-on-read:** convertir les données au moment de la lecture en un format exploitable
- **Append-mostly:** les mises à jour sont surtout immutables
 - modifications rares pour *optimisations ou RGPD*

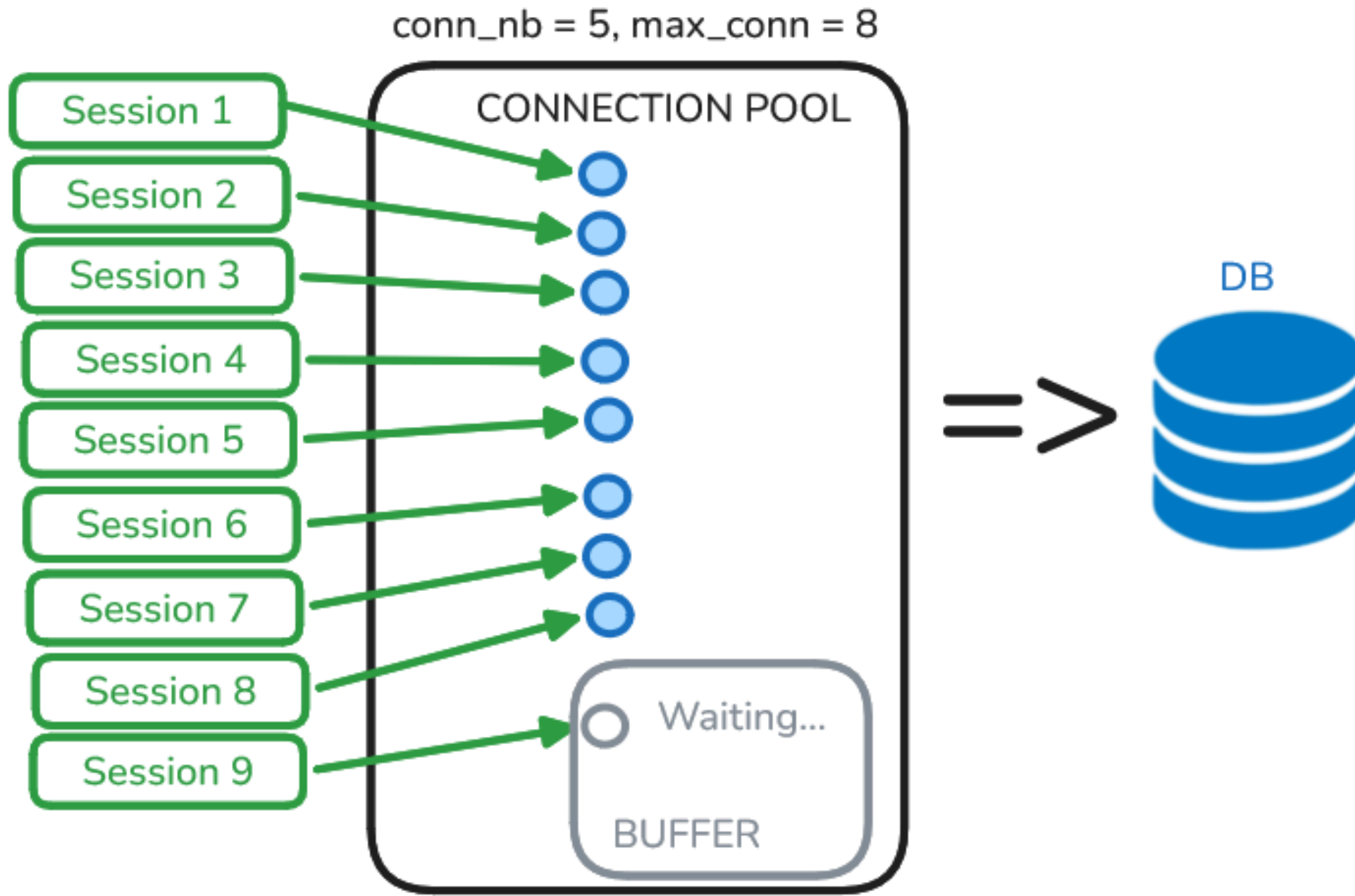
Data Lake: architecture



Pooling de connexions

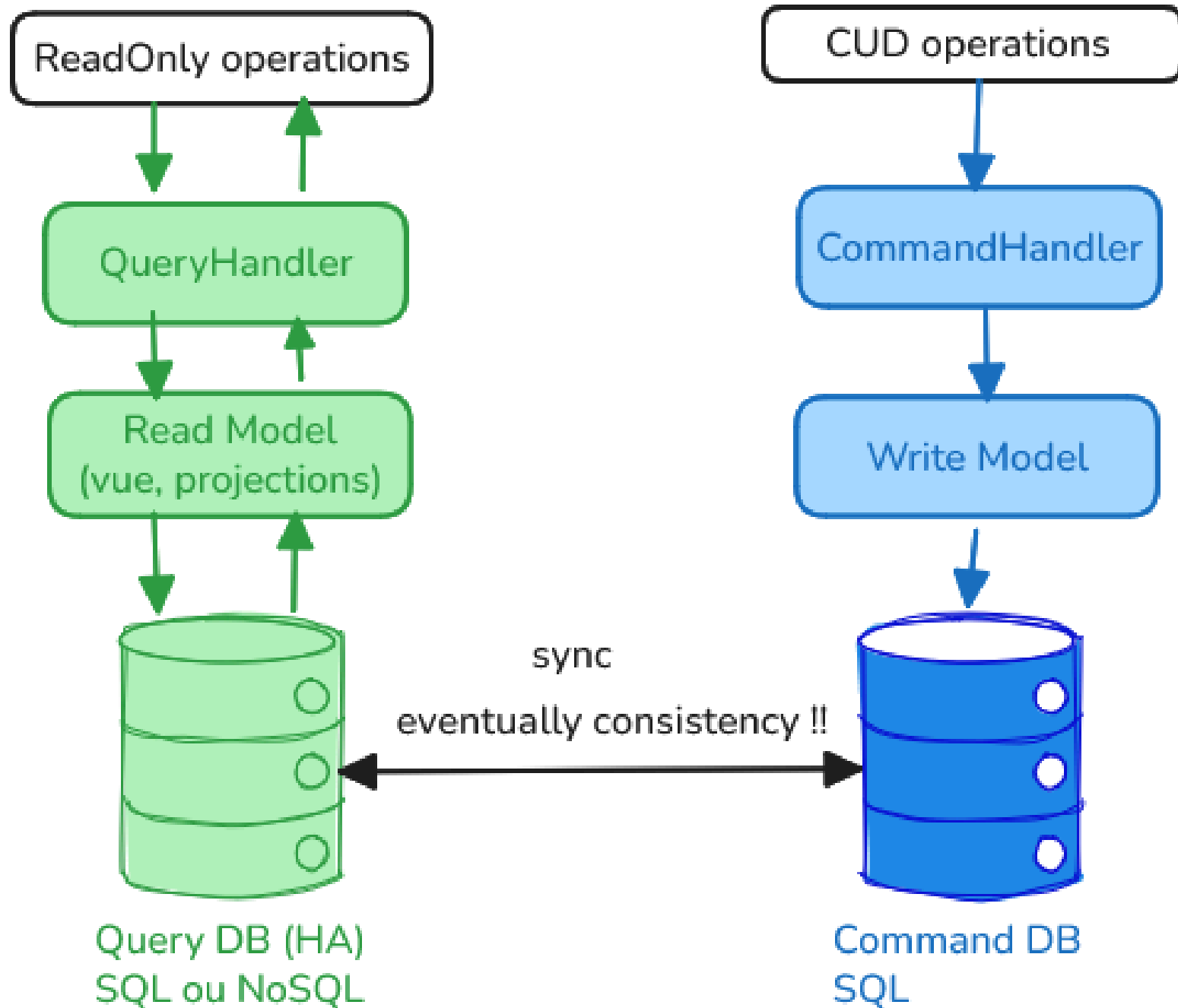
- préemption de connexions **toujours ouvertes** (actives + idle)
- *connexions temporaires* en cas de pics de trafic

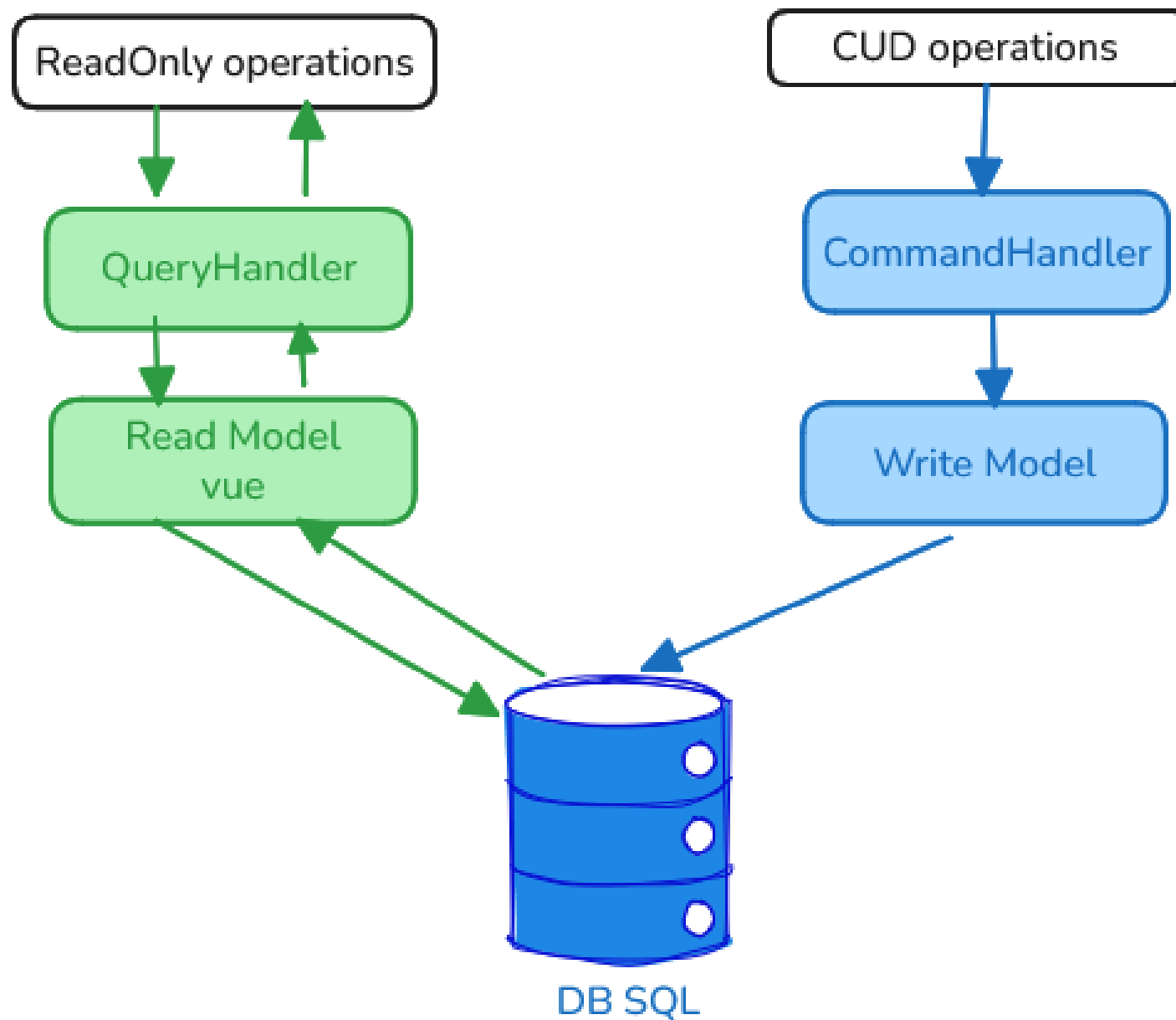




Command Query Responsibility Segregation (CQRS)

- séparation des modèles de données pour les **commandes** (écriture) et les **requêtes** (lecture)
- **+++** optimisation des performances de lecture et d'écriture
- **+++** intéressant pour les traitements complexes en écriture
- **---** : complexité accrue de gestion des données
- **---** : décohérence temporaire entre les modèles de données - *eventually consistency*





cohérence forte AVEC verrous R/W
MAIS latence en lecture OU Dirty Reads (verrous W)

Event Sourcing

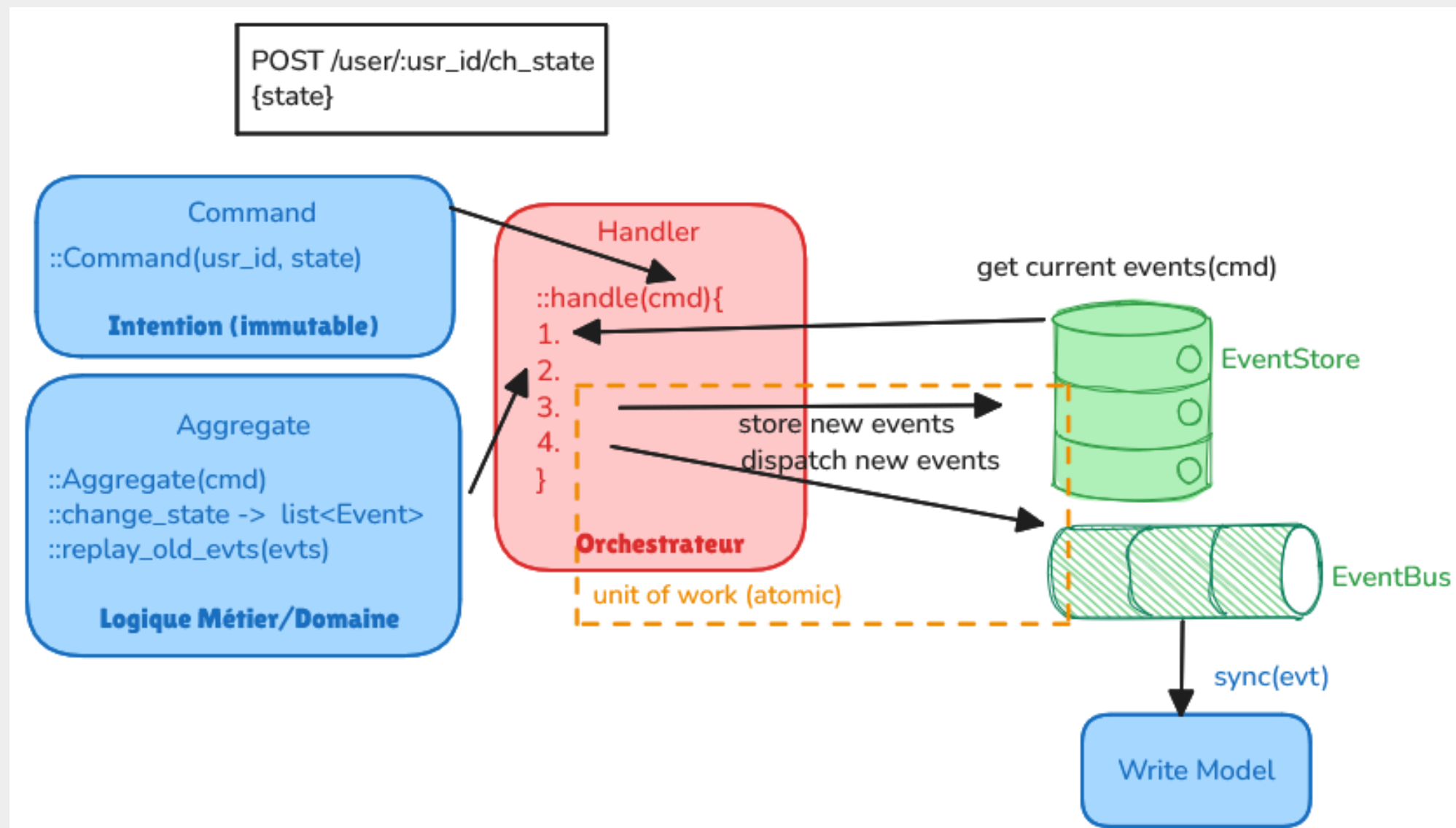
- chaque changement d'état d'une entité est enregistré comme un **évènement immuable** dans un *event store*
- l'état courant de l'entité n'est pas une valeur modifiée => mutable
 - mais la **reconstruction** de la séquence des évènements

+++ traçabilité et auditabilité

+++ rejouabilité des évènements

--- complexité accrue de gestion des évènements

CQRS + Event Sourcing



clients			commandes			
relation						
id	nom	email	id	client_id	montant	date
1	Alice	alice@ex.com	1	1	100.00	2024-01-15
2	Bob	bob@ex.com	2	1	50.00	2024-01-16
3	Carole	carol@ex.com	3	2	200.00	2024-01-17

- pour les systèmes *multi-utilisateurs et transactionnels*
- ACID
- rigides

ACID: propriétés

- **Atomicité**: jeux d'écritures en tant que *transactions*
 - TOUT (COMMIT) ou RIEN (ROLLBACK)
- **Cohérence**: respect des *contraintes d'intégrité* - unicité, cascades...
- **Isolation**: les transactions concurrentes ont le même résultat que si elles étaient exécutées séquentiellement, grâce à des mécanismes de *verrouillage ou de versionnage*.
- **Durabilité**: une transaction validée persiste même en cas de panne du système, grâce à des mécanismes de *journalisation*.

BDD: colonnes

```
## table orientée ligne:
```

```
id| nom    ,age, ville
```

```
1 | Alice ,30 , Paris
```

```
## table orientée colonne => juxtaposition de séries indexées
```

```
nom      | age    | ville
```

```
{Alice:1} | {30:1} | {Paris:1}
```

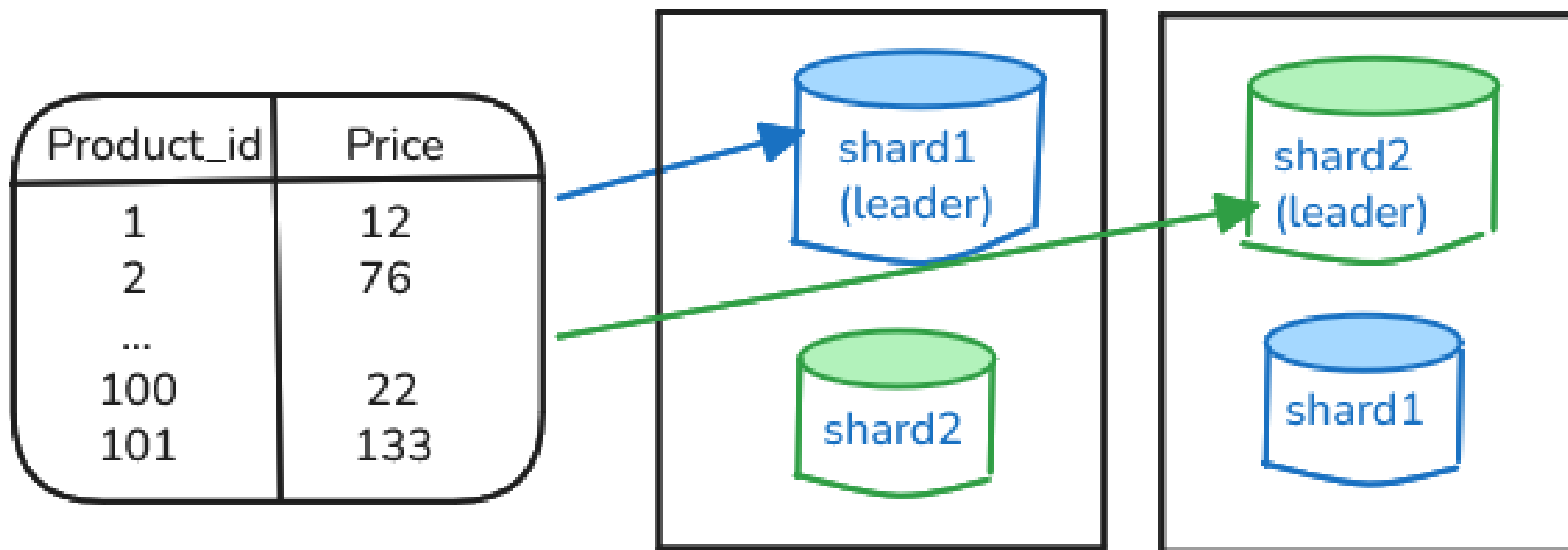
```
{Bob:2}   | {25:2} | {Paris:2}
```

```
{Carole:3}| {35:3} | {Lyon:3}
```

- ex: Apache Cassandra, HBase
- **calculs d'agrégats** massifs, **écritures rapides**, mais *pas de jointures*
- **Haute disponibilité / partitionnement**

colonnes: sharding (partitionnement)

- comme les colonnes sont indexées et sans jointures,
- on peut les répartir sur plusieurs nœuds physiques avec une clé de partitionnement (ex: **hash de l'id**) pour une *scalabilité horizontale*



BDD: documents

- gestion de données non/semi structurées et **hiérarchiques**
- format Binary JSON *BSON*, **lectures rapides**, mais *pas de jointures*

```
{
  "_id": "user123",
  "nom": "Alice",
  "commandes": [{
    "id": "order1",      # INDEX SECONDAIRE
    "montant": 100.00,
    "date": "2024-01-15",
    "items": [{ "produit": "Livre", "prix": 20.00 }]
  }],
  "tags": ["VIP", "Newsletter"]
}
```

BDD: séries temporelles

Timestamp	Sensor	Location	Temp	Humidity
2024-01-15 10:00	sensor1	Paris	22.5	60.0
2024-01-15 10:01	sensor1	Paris	22.6	59.8
2024-01-15 10:02	sensor1	Paris	22.4	60.2
2024-01-15 10:00	sensor2	Lyon	18.3	65.0

- Ex: Prometheus, InfluxDB
- pour les **données horodatées**
- algorithmes de compression spécifiques

BDD: vectorielle

- pour recherche sémantique de données non structurées (ex: texte, images, audio)
- RAG: **R**etrieval **A**ugmented **G**eneration: pour injecter des **connaissances d'entreprises** dans un LLM à partir d'une base de données vectorielle

TOKEN	Vector	
"chat"	[0.2, 0.8, 0.1, ...]	# Vecteur == Embedding ~ poids d'un LLM
"félin"	[0.21, 0.79, 0.12, ...]	# PROCHE
"voiture"	[0.9, 0.1, 0.7, ...]	# ELOIGNÉ

théorème CAP

“ une base de données distribuée ne peut garantir que 2 des 3 propriétés suivantes: ”

- **C**onsistency: tous les noeuds affichent les *mêmes données au même moment*
- **A**vailability: tous les noeuds actifs retourne une *réponse en succès à toute requête*
- **P**artition Tolerance: le système continue de fonctionner malgré les partitions réseau == *désynchronisation entre les noeuds*

exemples de compromis CAP:

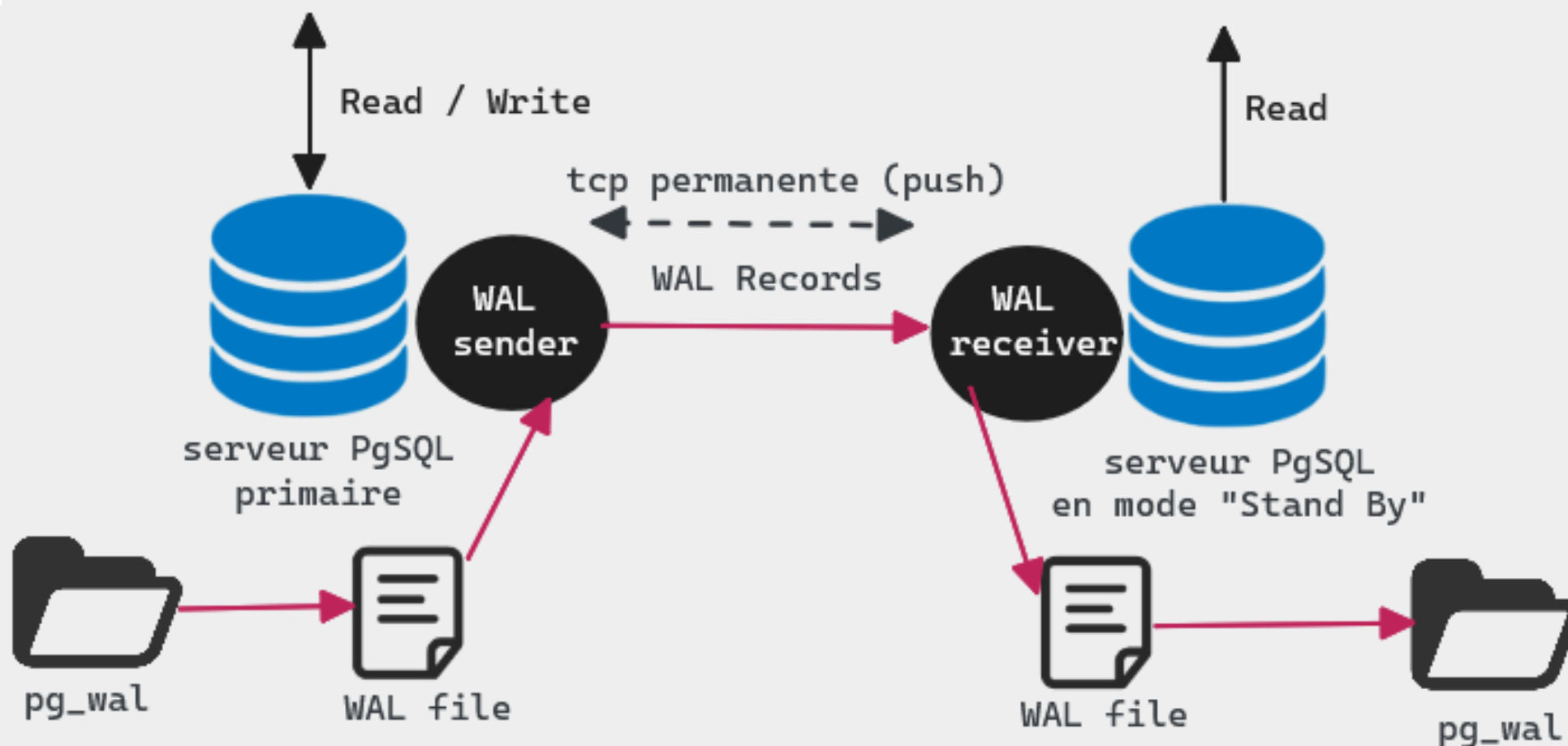
- **CA**: bases de données relationnelles classiques (ex: MySQL) sauf en cas de partitions réseau qui sont inévitables
==> IMPOSSIBLE EN PRATIQUE
- **CP**: bases de données NoSQL (ex: *MongoDB*) qui privilégient la cohérence au détriment de la disponibilité en cas de partitions réseau
- **AP**: bases de données orientées colonnes (ex: *Cassandra/CouchDB*) qui privilégient la disponibilité au détriment de la cohérence en cas de partitions réseau

III.7 Styles hybrides

- la plupart des architectures logicielles **combinent plusieurs styles**
 - ex: flux de données + évènementielle + agents + leader / follower
 - => *calcul distribué*

archi leader / follower

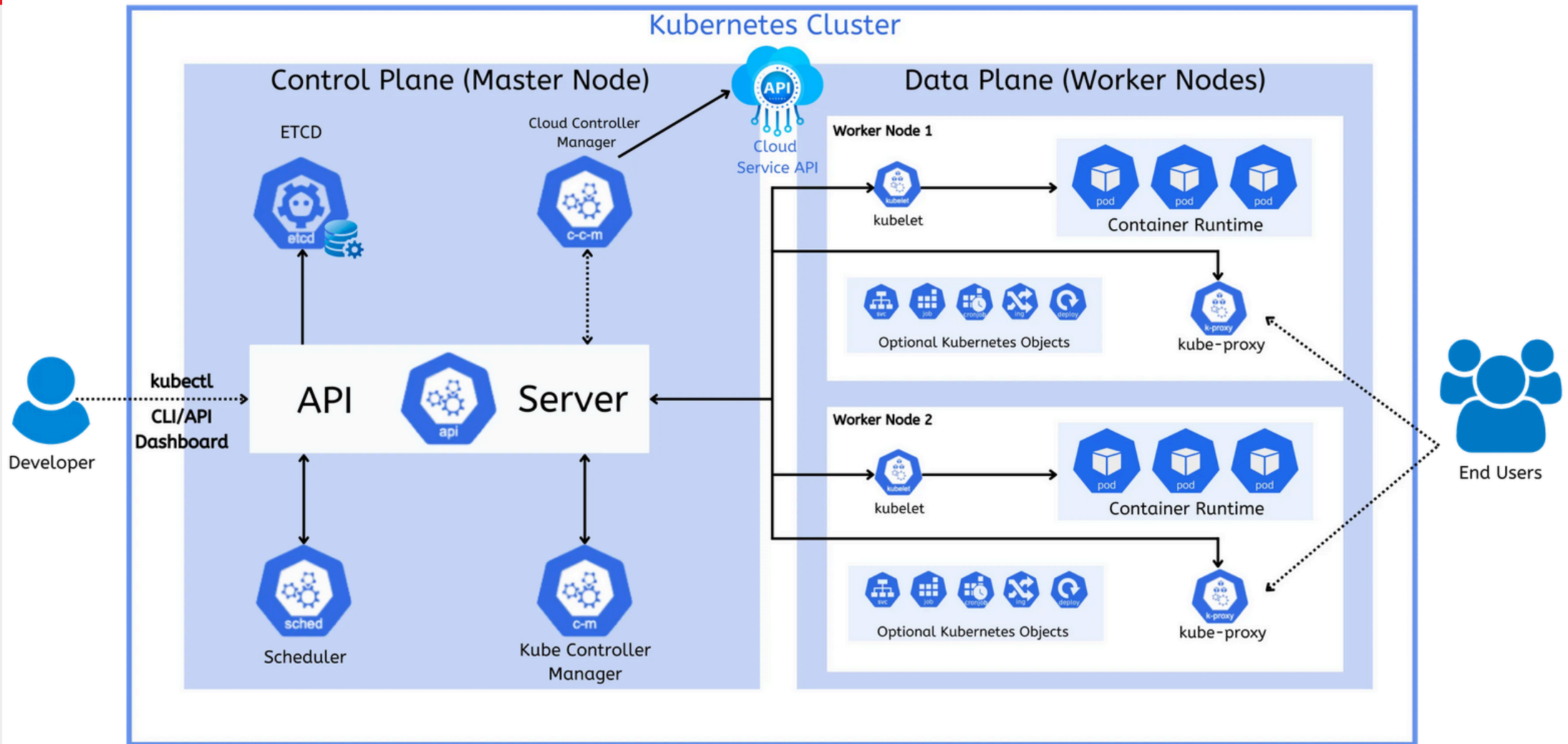
- un composant **leader** centralise les décisions
 - et distribue les tâches aux composants **followers**
- ex: **réplication "Statefull"** de *PostgreSQL*
- ex: **Kubernetes**
 - Gestion du *cycle de vie* des applications conteneurisée
 - *contrôle et optimise l'état* des applications souhaité par les développeurs



WAL: Write Ahead Logs

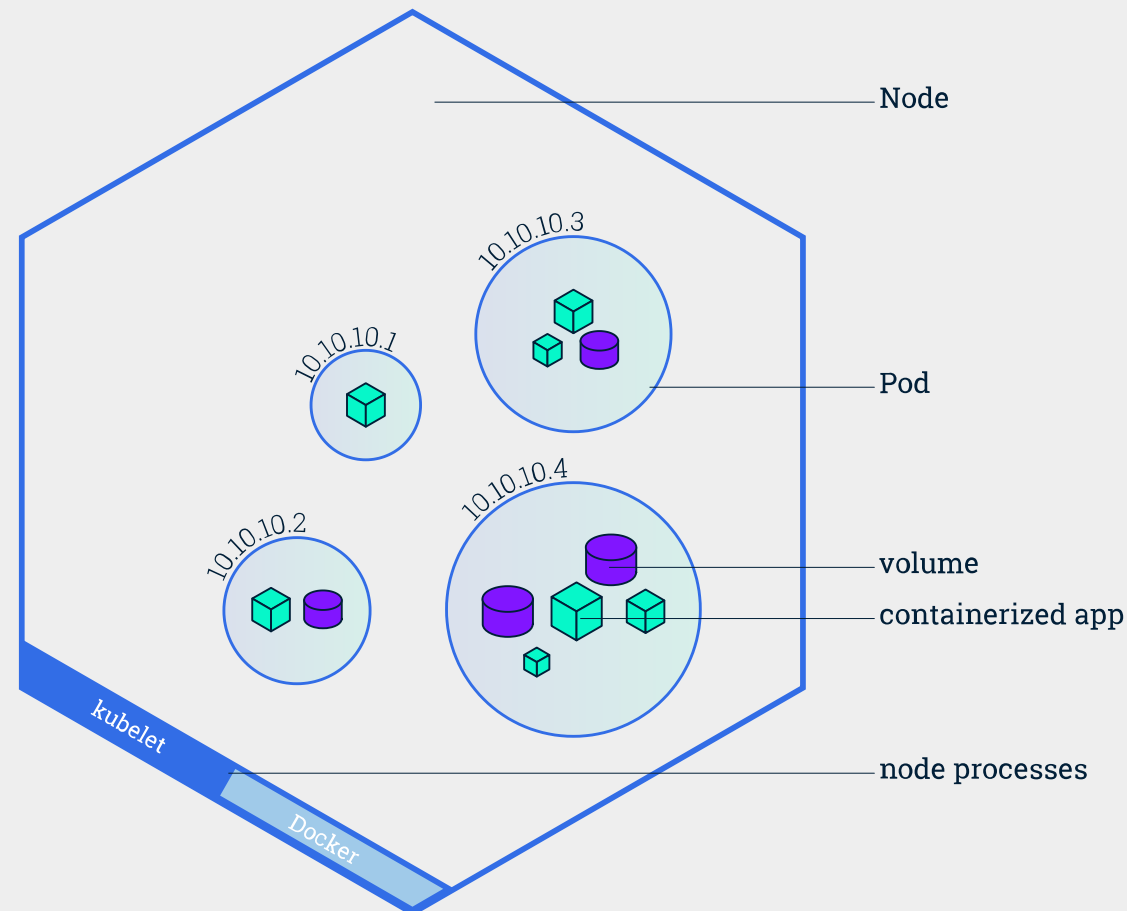
- journalisation des écritures avant exécution

Streaming: les enregistrements sont transférés en continu avant la fin du fichier

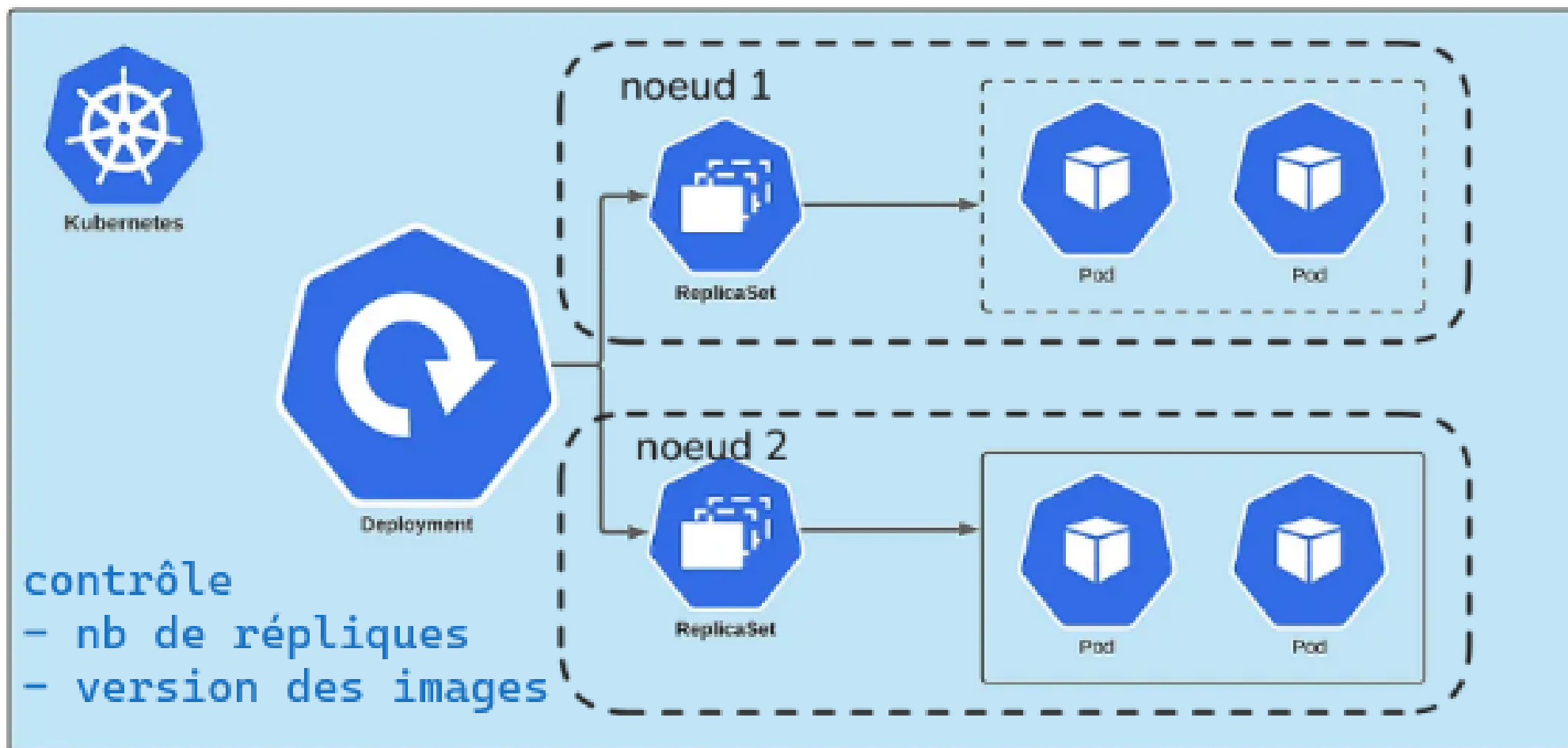


pod kubernetes

- processus partageant mêmes namespace et **points de montage** (volume)
- surtout, une **adresse IP commune**



Déploiement kubernetes



mise en réseau kubernetes

